



6CCS3PRJ

**Agent-based Modelling with the CAT Inequality as a Tool for
Epidemic Management in Emergency Departments**

BSc Computer Science with Intelligent Systems

Author: Vicente Sebastião

Supervisor: Dr Maria Polukarov

Student ID: 20069828

April 2023

Abstract

Research shows that individuals are especially prone to becoming infected by respiratory viruses in indoor environments. Research also shows that this risk is especially aggravated when the indoor environment is an Emergency Department (ED) during an epidemic. The use of Agent-based computer simulations for modelling the spread of viruses indoors can be a particularly useful tool for those responsible for managing these facilities. However, the most popular methods used to model the spread of airborne infectious diseases indoors, such as the Wells-Riley model, have fundamental disadvantages. Assumptions of well-mixed air, constant removal rate, uniform susceptibility and infectiousness may only sometimes hold in real-world scenarios. The Covid Airborne Transmission (CAT) inequality, developed by researchers at Johns Hopkins University, is a novel framework used to determine the risk of infection between hosts and susceptible individuals, overcomes certain assumptions of other models, and appears to fit particularly well with Agent-based simulations. However, the CAT inequality is yet to leave the paper and be applied to these simulations. This aim of this research is to develop an Agent-based simulation of an Emergency Room that takes advantage of the CAT inequality.

Originality Avowal

I verify that I am the sole author of this report, except where explicitly stated to the contrary. I grant the right to King's College London to make paper and electronic copies of the submitted work for purposes of marking, plagiarism detection and archival, and to upload a copy of the work to Turnitin or another trusted plagiarism detection service. I confirm this report does not exceed 25,000 words.

Vicente Monteiro Sebastião

April 2, 2023

Acknowledgements

I would like to thank Dr Maria Polukarov for her guidance. I would like to express my gratitude to her for providing me with the opportunity to work on this topic, and for her trust throughout.

Table of Contents

Abstract	i
Originality Avowal	ii
Acknowledgments	iii
Contents	iv
1 Introduction	1
1.1 Thesis	1
1.2 Motivation	1
1.3 Solution Objectives	2
1.4 Software Platform and Target OS	3
2 Background Research & Literature Review	5
2.1 Properties of Respiratory Viruses, Transmissibility, and Preventative Measures	5
2.2 Mathematics of Modelling Transmissions of Respiratory Viruses	8
2.2.1 Outdoor Models	8
2.2.2 Indoor Models: The Wells-Riley Equation	10
2.3 Agent-based Models for Simulation	12
2.4 The CAT Inequality.....	14
2.5 Agent-based Modelling Software	19
2.5.1 NetLogo with BehaviourSpace	19
2.5.2 Python 3 with Pygame, Pandas and Matplotlib Libraries	22
2.5.3 The Recursive Porous Agent Simulation Toolkit (Repast)	23

2.5.4	Swarm	24
2.6	Patient Flow in Emergency Departments	25
2.6.1	The Stavanger University Hospital Study	25
3	Requirements	28
3.1	Environment Requirements	28
3.2	CAT Inequality Environment Requirements.....	28
3.3	CAT Inequality Agent Environments	29
3.4	Additional Agent Requirements.....	29
3.5	Graphical User Interface (GUI) Requirements	29
4	Design, Implementation & Testing	31
4.1	Introduction	31
4.2	Design Process & Software Development Approach	31
4.3	Simulation Set-up.....	35
4.4	A* Pathfinding Algorithm.....	38
4.5	Main Loop	39
4.6	Rapid Tests at the Entrance	40
4.7	Movement sub-model	41
4.7	Contagion sub-model	50
5	Experimentation	55
5.1	Experimenting with Mask Parameters	55
5.2	Quantifying the impact of different quantities of hosts	57
5.3	Quantifying the impact of rapid-tests at the ER entrance.....	58
5.4	Quantifying the impact of social distancing measures and patient anxiety	59

5.5	Quantifying the impact of waiting times	61
6	Legal, Social, Ethical and Professional Issues	63
6.1	Introduction	63
6.2	Legal Issues	63
6.3	Social Issues	64
6.4	Ethical Issues	64
6.5	Professional Issues	64
7	Results/ Evaluation	66
9	Conclusion and Future Work	67
	References	68
	Appendix A- Experiments	72
	Appendix B- User Guide	76
	Appendix C- Source Code	80

Chapter 1

Introduction

1.1 Thesis

Using Agent-based models for simulation in combination with the CAT inequality framework is a helpful tool for evaluating resource allocation and intervention strategies in Emergency Departments for epidemic control.

1.2 Motivation

The motivation for the topic of this dissertation stems from several personal observations and experiences during the COVID-19 pandemic. It was evident to the author that some people would rather deal with specific health issues at home than go to a hospital during the pandemic. Hafner, K. (2020) After all, contracting a respiratory virus such as COVID-19 can aggravate other medical issues. Being in a hospital emergency department (ER) poses several risks to patients, such as entering into contact with other patients who are sick from infirmities, such as, but not limited to, respiratory viruses. (Chen, Y.-C., et al, 2004) At its peak, the COVID-19 pandemic was exuberating stress on ER units across nations unseen by most generations. (Gualano, M.R., et al, 2021) Those illest and needing hospitalization pose a contagion risk to everyone around them. A close family member of the author passed away after contracting the COVID-19 virus while hospitalized in 2022 for an unrelated reason. The death occurred after the patient was cleared and in the recovery room. It is not an isolated event. Reliable data provided by the NHS to The Guardian under freedom of information (FOI) laws in May 2021 indicate that 32,307 people were likely to or did become infected by COVID-19 in English hospitals since March 2020. Out of those, 8,747 have died. (NHS, as cited in The Guardian, 2021)

Furthermore, hospital ERs imposed with higher patient inflows due to the pandemic placed a strain on resources such as nurses and doctors and, in turn, have delayed many surgeries and appointments for unrelated ailments. (Kumar, D., Dey, T. 2020) Although there is significant literature on the spread of airborne respiratory viruses on the macro level, there is an increasing need to model situation-specific instances within indoor spaces. This work is motivated by the

idea that certain behaviours practised by patients and staff can be effectively detected and demonstrated to either increase or decrease the risk of virus spread in hospitals through simulation technology. Patient welfare is the central long-term objective and motivation of such a system. Simulation allows different hypotheses to be tested in a safe environment—the computer. Individual scientific observations on elements of virus behaviour paired with human compartments and the flow of people in an environment allow for multiple pieces of a complex puzzle to be put to the test simultaneously. From an economic point of view, if there is confidence in the simulation, costly safety measures can be tested before being implemented to demonstrate proof of concept.

1.3 Solution Objectives

The central focus of the solution is on the demonstration of the application of Agent-based models (ABM) to the field of mathematical modelling for infectious diseases indoors using the Contagion Airborne Transmission (CAT) model. To prove the thesis that ABM simulations are helpful for epidemiological management, the example chosen is the case of an Emergency Departments (ED) during an epidemic. The model should accurately reflect the impacts of certain measures taken by management to limit the propagation of the virus within the facility. Hence, the goal is that the end-users of such a system will be those tasked with improving the hospital ED services. The objective is to have a simulation that effectively shows the process of patients entering the ED and moving through the several stages until they are seen by a doctor. All parameters relating to the virus should be editable by the user as to see the impact of different scenarios on the viral spread to other patients within the facility. Further, there should be parameters that relate specifically to the hospital's logistics. A few examples of some parameters that the end-user should have access to include the utilisation of face-masks, the time each consultation takes, the time patients wait at the reception and if there are screening-tests to lead potentially infected patients through fast-track before they enter.

Potential Use Case / Scenario Example:

“A new strain of COVID-19 appears in India and virologists quickly study and gather the relevant data about the virus. In England, hospital management anticipate the spread of this new strain to the U.K. and use the developed system to predict the impact on the Emergency Department they run. They set all the relevant variables of the CAT inequality based on research conducted on this new strain into the simulation

parameters and then set certain values that represent their current operations such as how long the registration time takes for patients and how long the consultations are at the emergency department. Based on available data, they run the simulation assuming that for every minute there is a probability of 50% that a new patient will visit the ER and that 5% of those patients will be infected with the virus.

The simulation runs several experiments and the results indicate that this new strain is infecting considerably more people on average, ceteris paribus. The simulation experiments also indicate that increasing the mandatory mask policy from just any mask to 95% effective masks considerably lowers infections of other patients. Applying these measures, the hospital management is still concerned about this specific new strain, so they implement new policies for their resources such as decreasing consultation times in order to keep patients inside the building for less time. They also implement a self-registration system that patients can do on their phones to avoid the build-up of several patients in reception area. The hospital staff are finally happy with the simulation results. Sometime later the virus strain reaches the city where the ED is located and the hospital is appropriately prepared. However, they notice patients are oddly uneasy recently, perhaps due to external factors, and simply cannot sit still. They go back to the simulation and increase the patient anxiety parameter to match what they believe is the real-life equivalent. By running several experiments, they realise many patients are likely becoming infected due to excessive movement, so they implement a policy of screening patients for the virus at the entrance, and fast-track those with positive results straight to the consultations in order to avoid having these contact susceptible patients.”

This paper has the long-term goal to serve as an outline, or suggestion, for future academics to take an agent-first approach, meaning the disease is modelled as elemental parts of agents, in order to develop models for simulating the spread of airborne viruses, and potentially other diseases.

1.4 Software Platform and Target OS

Such a system can be designed using several software platforms and be made able to run in different target operating systems depending on the intended control, complexity, and ease of

use desired by the software engineers. In section 2.5, there is an outline of the thought process that led to the choice of the software platform and target operating systems for this specific simulation system.

This system is developed in the software platform NetLogo 6.3.0. Moreover, it can be run in the operating systems of Linux, Windows, and MacOS throughout several of their versions.

Operating system compatibility testing was conducted using the Oracle virtual machine VirtualBox software to simulate several operating system versions. Windows is the most common operating system and most commonly used by government employees due to its widespread familiarity therefor, an additional focus was concentrated on testing all its versions. Due to the extensive range of Linux distributions, only one version was tested. Ubuntu 10.04 was the selected, tested Linux version. Further, MacOS versions are not reliably virtualized and are only meant to run on Apple machines; hence only version 12.5 was tested, given it was this author's machine version at the time and also the operating system used during the development process. Below is a table which compares expected system requirements provided by NetLogo's developers, Northwestern University, and the author's findings during testing.

<i>Windows</i>	<i>NETLOGO 6.3.0 COMPATABILITY</i>	<i>Test Results</i>
Windows 11	Up to Date	Fully Functional
Windows 10	Up to Date	Fully Functional
Windows 8	Up to Date	Fully Functional
Windows 7	Up to Date	Fully Functional
Windows Vista	Up to Date	Fully Functional
Windows XP	Up to NetLogo 5.2.1	Functional, but there might be bugs.
Windows 2000	Up to NetLogo 5.2.1	Functional, but there might be bugs.

<i>MacOS</i>	<i>NETLOGO 6.3.0 COMPATABILITY</i>	<i>Test Results</i>
MacOS ≤ 10.8.3	Up to Date	Fully Functional on MacOS 12.5
MacOS 10.7	Up to NetLogo 5.2.1	Not Tested
MacOS 10.6	Up to NetLogo 5.2.1	Not Tested
MacOS 10.5	Up to NetLogo 5.1.0	Not Tested
MacOS 10.4	Up to NetLogo 5.1.0	Not Tested

<i>Linux</i>	<i>NETLOGO 6.3.0 COMPATABILITY</i>	<i>Test Results</i>
Debian Distributions	Up to Date	Fully Functional in Ubuntu 10.04 Lucid Lynx

Chapter 2

Background Research & Literature Review

In this chapter, we discuss Agent-based modelling as a form of simulation and how it is applied throughout the software industry for similar purposes. We then go into detail regarding the other aspect of this project, which involves the state of art of modelling of infectious viruses through mathematics. We finally look at the research regarding patient flow within emergency departments, which helps us realistically bring the entire system together.

2.1 Properties of Respiratory Viruses, Transmissibility, and Penetrative Measures

In order to develop an appropriate ABM simulation of person-to-person viral transmissions within an indoor environment, we must understand the inherent properties of respiratory viruses. The scope of this work focuses on viruses that primarily cause lower-tract respiratory infections; henceforth, the term "respiratory virus" will refer to these specifically. These viruses are generally more severe to humans; therefore, their epidemics pose a greater risk to society. They are distinct from the well-known rhinoviruses. Rhinoviruses, also known as the cause of a common cold, are viruses that typically infect the upper respiratory tract (nose, throat, and sinuses). Across three main species of rhinoviruses, there are over 165 types known to humans. It is highly contagious and transmitted through direct contact with an infected person through aerosols or respiratory droplets or by touching surfaces contaminated with virus fomites. Symptoms of rhinovirus infections can include a runny or stuffy nose, sore throat, cough, and mild fever. Generally, colds are not considered serious and usually resolve independently within a week or two. (Jacobs, Samantha E., et al., 2013)

On the other hand, a lower-tract respiratory virus is a type of virus that can infect the entire respiratory system, including the lungs. Respiratory viruses include influenza, coronaviruses (including COVID-19), respiratory syncytial (RSV), and others. These viruses are often more

serious than cold viruses and can cause severe respiratory symptoms, such as coughing, shortness of breath, and pneumonia.

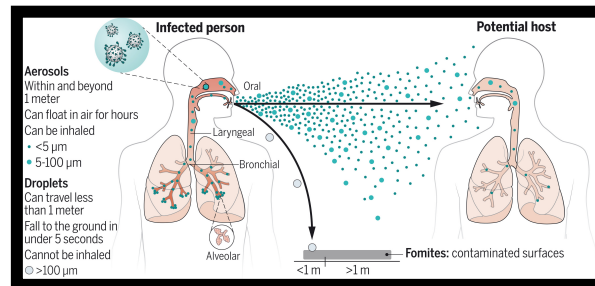


Figure 1: 'Phases involved in airborne transmission of respiratory viruses.' (Carry,2021)

Nevertheless, this work demonstrates the use of CAT inequalities described in section 2.1.5 in conjunction with ABMs. The CAT inequality is supposed to be adapted to fit any respiratory virus transmitted through aerosols and respiratory droplets.

According to (Leung, N.H.L. 2021), viruses are transmitted via airborne respiratory droplets and aerosols. Figure 1 above illustrates respiratory droplets passed from one individual to another with COVID-19 particles. As explained by Leung, aerosols are tiny airborne particles on the scale of microns that can remain suspended in the air for extended periods. When an infected person talks, coughs, or sneezes, these aerosols can contain the virus and travel through the air, potentially infecting others inhaling them. Aerosols are small enough to travel further distances and can also be carried by air currents or ventilation systems. Droplets are larger particles that are expelled from an infected person's mouth or nose when they talk, cough, or sneeze. These droplets can also contain the virus and travel a shorter distance before falling to the ground or onto surfaces. However, if someone is close enough to the infected person, they can still be exposed to the virus through these droplets. The human respiratory tract produces respiratory droplets, either thoracic or extrathoracic, with potentially varied pathogen loads, when an infected person releases them by lung-fluid fragmentations (Bourguiba, et al., 2014 as cited in Drossinos, Y., Stilianakis, N.I.,2020).

Transmission of pathogens through indirect means is a fascinating topic. There are several research pieces on the risk posed by virus-laden surfaces. In one study, the authors reviewed the evidence that 'contaminated surfaces contribute to the transmission of hospital pathogens'. (Otter, J.A., Yezli, S., Salkeld, J.A.G. et al., 2013) Although the range of virus species studied

was diverse, staphylococcus aureus, including MSRA, a virus which causes respiratory tract infections, was found to have the ability to survive on surfaces from a period of 7 days to 12 months (Otter et al., as cited in Wagenvoort, J.H.T et al. 2000). The authors then cite research which determined that through indirect contact, virus laden surfaces, 40% of health care personal in a test group of 50 staff were found to have acquired MSRA on their gloves. (Hayden, M.K., Blom, D.W., Lyle, E.A., Moore, C.G. and Weinstein, R.A., 2008) Another study found that '3 of 26 patients acquired MRSA directly from contaminated environmental surfaces'. (Hardy, K.J., et al, 2006 as cited in Otter, J.A., et al, 2014).

Nevertheless, many studies examined suggest that different viruses behave differently on surfaces and pose different risks. The works of both Hayden, et al, 2008 and Hardy, K.J.,2006 suggest that contaminated surfaces pose a significant risk to humans in the case of MSRA, yet more recent studies on COVID-19 claim that the risk of contracting the virus through fomites is negligible. For example, in a study it was estimated that the risk of being infected by touching a contaminated surface 'ranged from 2 in 10 million to 4 in 10 thousand'. (A.P. Harvey, et al, 2020) The study's findings are compelling as the researchers used quantitative microbial risk assessment, a well-established mathematical modelling method used to determine the risk of virus infection through contact with an environment. The data used in the study was also ample as the researchers used swab samples on over thirty surfaces in publicly used spaces such as post offices and metros. Two other studies, with agreeing results, conclude that the risk of infection via fomites for Covid-19 is exaggerated and negligible (Emanuel Goldman, 2020), (A.K. Pitol, T.R Julian 2020). In conclusion, the role of fomites in the transmissibility of respiratory viruses is still debated, and there needs to be more knowledge on which viruses pose a significant threat and which do not.

There is an abundance of research published on the matter of epidemics and the strategies to reduce the spread of airborne respiratory viruses. For example, in the Guideline for Preventing Transmission of Infectious Agents in Healthcare Settings from the CDC, there is excellent detail on practical measures. (Siegel, J.D., et al., 2007-2022)

Part II of the CDC guideline outlines the 'fundamental elements needed to prevent infections in healthcare settings'. Administrative measures, education for patients, hand hygiene, personal protective equipment (PPE), patient placement, safety work practices by members of staff and patient placement are all sub-chapters of the guideline relevant to the development process of

the ABM simulation. The ease with which these measures can be implemented into ABMs will vary greatly. While personal protective measures such as face coverings have significant literature, such as in a study where the authors test six kinds of popular face masks and their efficacy against COVID-19 (Asadi, S., et al., 2020), other measures such as the education of patients, and if staff are practicing appropriate safety measures, are inherently less quantifiable.

2.2 Mathematics of Modelling Transmissions of Respiratory Viruses

In this section, we cover literature about transmitting airborne diseases, including the chosen approach of the Agent-based modelling simulation. The models used to study indoor virus transmissions between people differ considerably from those used to study virus transmissions across large spaces such as cities, countries or the entire world. When modelling how a virus spreads indoors, the focus is on how the virus spreads through the particular environment, possible measures to prevent it and how many people get infected given the defined parameters. Indoor models are used to find out strategies to reduce transmissions in a very particular place, and they are used to measure the usefulness of each intervention. Therefore, simulations of indoor viruses often focus on high-density scenarios, such as offices, schools, and public transportation. However, when modelling virus transmissions in large areas, such as throughout a country, we are typically interested in how the virus spreads through human-to-human contacts, such as through social interactions, travel, population density, the density of green spaces, or even the distribution of population centres. Due to their differences, the mathematical approaches taken to each scenario are different.

2.2.1 Outdoor Models

The SEIR Model is the most common mathematical framework for modelling the spread of viruses outdoors. (Wu, Y., Sun, Y. and Lin, M., 2022). The model divides the population into different compartments, including susceptible, exposed, infected, and recovered individuals. It can be used to study the impact of different interventions, such as social distancing, on the spread of the virus.

The SEIR model is a system of ordinary differential equations that describes the flow of individuals between four compartments (Hethcote, H.W., 2000) (Keeling, M.J. and Rohani, P., 2008):

susceptible (S), exposed (E), infectious (I), and recovered (R).

The equations governing the model are:

$$dS/dt = -\beta SI$$

$$dE/dt = \beta SI - \alpha E$$

$$dI/dt = \alpha E - \gamma I$$

$$dR/dt = \gamma I$$

Where β is the transmission rate, α is the rate at which individuals move from the exposure to the infectious compartment, and γ is the recovery rate. The first equation represents the rate of change of the number of susceptible individuals over time, which is proportional to the product of the number of susceptible and infectious individuals while the second equation represents the rate of change of the number of exposed individuals over time, which is the difference between the rate at which individuals move from the susceptible to the exposed compartment and the rate at which individuals move from the exposure to the infectious compartment. The third equation represents the rate of change of the number of infectious individuals over time, which is the difference between the rate at which individuals move from the exposed to the infectious compartment and the rate at which individuals move from the infectious to the recovered compartment. The fourth equation represents the rate of change of the number of recovered individuals over time, which is proportional to the number of individuals who recover from the disease. (Ma, S. 2009). The SEIR model assumes that individuals in the susceptible compartment become exposed to the disease at a rate proportional to the product of the number of susceptible and infectious individuals. Once exposed, individuals move to the infectious compartment after a latent period of $1/\alpha$. Infected individuals remain infectious for an average of $1/\gamma$ before recovering and moving to the recovered compartment. The SEIR model can be used to simulate the spread of infectious diseases in a population and to evaluate

the impact of different control measures, such as vaccination or social distancing, on the dynamics of an outbreak. By adjusting the model's parameters, researchers can estimate the likely course of an outbreak and identify the most effective interventions to control its spread. (Ma, S. 2009).

However, it is important to note that the SEIR model is a simplified representation of reality, and some assumptions underlying the model may not hold in practice. For example, the model assumes that individuals mix randomly within the population, which may not be the case in reality. Additionally, the model assumes that the transmission rate, recovery rate, and other parameters remain constant over time, which may not be accurate for all diseases. (Ma, S. 2009).

Therefore, it is important to use the SEIR model in conjunction with other data sources and modelling techniques, such as data from contact tracing, genomic sequencing, and agent-based models, to improve the accuracy of the predictions and inform public health policy decisions (Ferguson et al., 2020).

2.2.2 Indoor Models: The Wells-Riley Equation

The Wells-Riley model is a mathematical model used to estimate the risk of airborne transmission of infectious diseases in indoor environments. The model assumes that infectious particles are released into the air by infected individuals and then inhaled by susceptible individuals. (Gammage, R.B. 1996).

The Wells-Riley model uses a mass balance equation to estimate the concentration of infectious particles in a room over time. The equation is given by:

$$dC/dt = Q/V * (C_0 - C) - \lambda * C$$

where C is the concentration of infectious particles in the air (particles/m³), t is time (s), Q is the ventilation rate (m³/s), V is the volume of the room (m³), C_0 is the concentration of

infectious particles emitted by infected individuals (particles/ m^3), and λ is the removal rate of infectious particles from the air (s^{-1}). (Gammage, R.B. 1996).

The first term on the right-hand side of the equation represents the rate of change of the concentration of infectious particles due to ventilation. It assumes that the concentration of infectious particles is proportional to the concentration of emitted particles ($C_0 - C$), and that the ventilation rate is the rate at which air is exchanged in the room. The second term represents the rate of change of the concentration of infectious particles due to removal. It assumes that the concentration of infectious particles decays exponentially over time with a removal rate λ . The Wells-Riley model can be used to estimate the probability of infection for susceptible individuals in the room, given the concentration of infectious particles and the duration of exposure. The probability of infection is given by:

$$P = 1 - \exp\left(-\frac{Q}{V} * C * t\right)$$

The variable t is the duration of exposure (s). This equation assumes that the probability of infection is proportional to the concentration of infectious particles and the duration of exposure. (Gammage, R.B. 1996).

One of the main limitations of the Wells-Riley model is its assumption of well-mixed air in the room. This assumption means that the concentration of infectious particles is uniform throughout the room, which may not always be the case in real-world scenarios. In reality, airflows in indoor environments can be complex and may lead to spatial variations in the concentration of infectious particles. (Sze To, G.N., Chao, C.Y.H., 2010) For example, the concentration of infectious particles may be higher near an infected individual or in poorly ventilated areas of the room. As a result, the Wells-Riley model may underestimate or overestimate the risk of infection in some regions of the room. Another limitation of the Wells-Riley model is its inability to account for the effects of different ventilation systems or air-cleaning technologies. The model assumes that ventilation is the only factor affecting the concentration of infectious particles in the air and that the removal rate of infectious particles is constant over time. However, the effectiveness of ventilation and air cleaning technologies can vary depending on several factors, such as the type of system, the infected individual's location, and the infectious particles' size and distribution. For example, some ventilation systems may be more effective at removing infectious particles near the emission source, while

others may distribute the particles evenly throughout the room. Similarly, air cleaning technologies may have different efficiencies for different types and sizes of particles. (Sze To, G.N., Chao, C.Y.H., 2010) Furthermore, the Wells-Riley model assumes that all individuals in the room have the same susceptibility to infection and that the infectious particles are equally infectious to all individuals. However, in reality, the susceptibility and infectiousness of individuals can vary depending on several factors, such as age, health status, and immunization. For example, young children, elderly individuals, and individuals with weakened immune systems may be more susceptible to infection. In contrast, asymptomatic individuals or individuals with mild symptoms may be less infectious than symptomatic individuals. Lastly, the Wells-Riley model assumes that the infectious particles are airborne and can remain suspended in the air for an extended period. However, some infectious particles may settle on surfaces or be removed from the air by other means, such as filtration or deposition. The Wells-Riley model does not account for these processes, which may affect the room's concentration and distribution of infectious particles. (Sze To, G.N., Chao, C.Y.H., 2010)

In conclusion, the Wells-Riley model is a valuable tool for estimating the risk of airborne transmission of infectious diseases in indoor environments. However, it has several limitations that should be considered. The assumptions of well-mixed air, constant removal rate, uniform susceptibility and infectiousness may only sometimes hold in real-world scenarios. The model may not account for the effects of different ventilation and air cleaning systems.

2.3 Agent-based Modelling for Simulation

Agent-based modelling (ABM) is a computational technique, and a subfield of Artificial Intelligence, that uses dynamic systems of interacting agents to simulate complex systems. (Bonabeau, E. 2002) Although there are several interpretations and descriptions concerning the fundamental properties of "agents", the theoretical consensus amongst scholars is that these are autonomous, hence independently impactable, goal-oriented entities that interact with each other in an environment. (Nigel, G. , Klaus, T. 2005) Agents are sometimes referred to as objects with personalities or attitudes. However, while they share similarities to objects such as information hiding and encapsulation, they go beyond objects with features such as inferential capabilities, adaptability and mobility. (Bonabeau, E. 2002) Most importantly, an object must be called by other system components to function independently of how complex

it may be. However, an Agent can be independent of the system in itself, and this is also known as the property of emergence. Emergent behaviour is a hallmark of complex systems and is often difficult to predict or explain using traditional analytical methods. In ABM, emergence is captured through the interactions between agents and their environment, which can lead to patterns, structures, and dynamics not present at the individual level. (Louis Wang, 2002)

ABM has a rich history dating back to the early days of computer simulation and has been applied to a wide range of problems, from ecology to economics. (Railsback, S.F., Grimm, V. 2011). ABM has become an increasingly used method in recent years given the widespread concern for airborne viruses ranging from avian flu cases to the Covid-19 pandemic. The basic idea behind ABMs for viral propagation in populations is to model individual agents as humans and, in some cases, animals, each of whom can potentially contract and spread the virus. These agents can handle the virus differently. The interactions between these agents and their environment are then modelled using various techniques, such as network theory or spatial modelling. (Bonabeau, E. 2002) One of the advantages of ABM is that it can capture the complexity of real-world systems that are difficult to model using traditional mathematical techniques. This is particularly important for modelling the spread of viruses, which involves many factors such as human behaviour, mobility patterns, and social networks. By modelling individual agents and their interactions, ABM can provide a more detailed and realistic picture of how a specific virus might spread and assist in determining the aggravating factors. ABM for virus spread can also be used to explore "what-if" scenarios, allowing researchers to test the effectiveness of different intervention strategies. In this paper, we take a look at how certain factors, such as mask effectiveness, hospital flow, resistance to infection, and viral properties, impact the health and safety of a hospital emergency room.

In an Agent-based modelling simulation of airborne virus spread, agents get infected through contact with infectious agents. The exact mechanism for infection can vary depending on the specific model, but the general process involves the following steps: The simulation begins with a population of agents, each with its own characteristics, such as age, gender, location, and health status. Some agents are designated as infectious, meaning that they carry the disease and can transmit it to others. The infectious agents are assigned a level of infectiousness, determining the transmission probability per contact. (Brauer, F., Driessche, P. van den, Wu, J. 2008) The simulation proceeds in discrete time steps, during which agents move and interact with each other based on their behaviour rules and the environment. When two agents come

into contact, there is a probability of transmission based on the infectiousness of the infectious agent and the susceptibility of the susceptible agent. If the transmission occurs, the susceptible agent becomes infected and is assigned a period during which they are infectious. The simulation continues with infected agents transmitting the disease to susceptible agents in subsequent steps until the disease spreads throughout the population or the epidemic is contained through interventions. (Bonabeau, E. 2002) The exact rules for contact and transmission can vary depending on the specific model and the disease being simulated. For example, some models may allow airborne transmission, while others may only allow direct contact transmission.

The usual approach, as described, uses an assigned probability to determine if an infected agent passes on the pathogen to the susceptible agent upon their contact at a given proximity. This research focuses on improving the existing transmission model, which is oversimplified, and taking full advantage of the capabilities of agents by replacing a simple probability with an innovative inequality which fits in with the properties of agents. The following section outlines the relevant literature.

2.4 The CAT Inequality

The Covid Airborne Transmission (CAT) inequality was developed by researchers from John Hopkins University and the University of Mississippi in 2020 as a framework for determining the risk of airborne transmission between a COVID-19 host and a susceptible person in a given environment. (Mittal, R., Meneveau, C. and Wu, W. 2020) All the literature provided under this sub-chapter regarding CAT inequality is credited to the stated authors. Additional background will be provided for the scope of this work on the applications of the given inequality as a critical part of the ABM system. Although the given name of the inequality at hand has the term "COVID" in it, there is no evidence that this framework cannot be applied generally to other similar respiratory viruses, given that none of the variables in the formula pertains specifically to COVID-19. The opposite is in fact the case, as it can be understood that the variables pertain to biological factors that contribute to the viral transmission of respiratory viruses in general. Throughout the research of Mittal et al., when in doubt of possible values for parts of the inequality, the researchers looked to the literature on other viruses, such as Influenza, to fill in the missing information. Hence, not even the original authors' work could represent COVID-19 immaculately. Likewise, this research focuses on the applications of

inequality to ABMs, rather than on the details of virology. Hence, the data used in the implementation and testing sections for the simulation will be plausible and mostly pertain to a virus similar to COVID-19, given that it has the most easily accessible data for each variable but is not intended to represent any particular virus. Once the potential for this method is demonstrated, it will be down to virologists to determine the values for each variable in future research for each virus they wish to simulate.

Until now, there have been no known applications of the CAT inequality to an Agent-based simulation which motivated us in this research to use this specific method to model the transmissibility of the virus within the hospital emergency room. The authors of the study claim to have been inspired by the Drake Equation which provides a simple formula to determine something very complex like ‘the number of advanced life forms that exist elsewhere in our galaxy.’ Although it might initially appear far-fetched to adapt what is essentially an alien detecting formula to the arguably more serious application of healthcare, it turns out the CAT inequality is structured perfectly for Agent-based Modelling.

The CAT Inequality:

$$(R_h \times f_{vh} \times f_{mh} \times f_{ah}) \times (f_{at} \times f_{vv}) \times (f_{is} \times f_{ms} \times T_s) \geq N_{ID}$$

Where:

$$(R_h \times f_{vh} \times f_{mh} \times f_{ah}) = \text{'The Host Dependent Variables'}$$

$$(f_{at} \times f_{vv}) = \text{'The Enviornment Dependent Variables'}$$

$$(f_{is} \times f_{ms} \times T_s) \geq N_{ID} = \text{'The Suceptible Dependent Variables'}$$

The CAT inequality is in its essence a mere combination of nine terms multiplied together to form a product which is balanced against the other side of the inequality. If the product exceeds a certain value, making the inequality true, then a subject is deemed infected with the virus. What is particularly interesting is the fact that the nine terms are divided into three categories, four terms for the host dependent variables, two terms for the environments and four terms for the susceptible dependent variables. This means that in the context of Agent-based modelling within a simulation, there can be a clear attribution of each part of the inequality to the infected agents, the susceptible agents and the environment. The inequality is nonetheless slightly more

complex than plugging in a variable for each term as some terms are simplified. Further, the authors of the CAT Inequality themselves acknowledged that there was not enough information currently available to provide possible values for all the terms, which leaves a lot of room for further research to take place.

The variable R_h is equivalent to the ‘rate of expulsion of respiratory droplets from the nose and mouth of the host’ (Mittal et al, 2020). The value is measured in the unit of millilitres per second ($\frac{mL}{s}$). The authors break down R_h into the product of two other newly introduced variables, V_{Eh} and N_{dh} , where V_{Eh} is the ‘volume expiration of the host’ and N_{dh} is the number of ‘droplets per unit of volume’, therefor density of the exhalation of the host. According to research (Morawska, L., 2009 as cited in Mittal, et al 2020),

‘The measured values of N_{dh} for breathing are about $0.1 mL^s$ and this suggests that for R_h breathing could range from about 10 to 200 droplets per second depending on the ventilation rate’

We can therefore calculate that the possible range for R_h is between (100×0.1) to (2000×0.1) , which is equivalent to 10 to 200 droplets per second given the provided research. Nevertheless, the rates for activities other than normal breathing, such as talking or exercising will increase the value of R_h greatly. According to recent research, the value of R_h could be between grander by a factor of two eight times if the person is talking, and the value increases with loudness. The louder a person is, the more emissions tend to come out of their mouth. (Sima Asadi, et al, 2019 as cited in Mittal et al, 2020) This means that in an ABM simulation, there might be a need to control the R_h variable of the Host agents depending on if they are engaging with other agents or not. Finally, research indicates that in the rarer event that a person is classified as a “super producer”, then the value of R_h could be much higher, and the risk could be increased by a ‘factor of 1000 or more’ given that these super producers ‘generate droplets at rates that are 10 or more times higher than others’ (Edwards, D.A., et al, 2003, & Lindsley, W.G., et al, 2012 as cited in Mittal et al, 2020)

The variable f_{vh} ‘represents the fractional viral emission load’(Mittal et al, 2020) which is in essence the mean quantity of viral particles in each droplet that is exhaled by the host of the virus. Given that the authors of the CAT inequality did not find any promising data pertaining to this variable applied to COVID-19, they relied on data form other studies on aerosol

emission properties in patients with respiratory infections such as quantity, size and distributions to come up with a lower bound estimate of 0.5% or $f_{vh} = 0.005$. (Lee, J., 2019, as cited in Mittal et al, 2020)

The variable f_{mh} is a measure for how many droplets get past any personal protective equipment the host might be wearing. This variable is particularly useful in simulations, as it allows us to test the efficacy of different masks while keeping all other variables constant. The value inserted into this variable is hence the inverse of a mask's efficacy. For example, if a mask is 70% effective, meaning 30% of virions get through, then f_{mh} would be equal to 0.30.

The variable f_{ah} is the final host related variable. It represents 'the fraction of expelled droplets that aerosolize.' (Mittal et al, 2020) This variable is highly dependent on factors such as if a person is speaking, exercising or simply breathing and there is research which indicates that the more intense the activity, the more particles will aerosolize. However, more research has to be conducted to determine the exact values as there is a lot of scientific debate on the matter. Research shows that the larger particles fall to the ground quickly, and the smaller a particle is the more likely it is to aerosolize. The thresholds are severely debated as to what exactly constitutes a large or small particle, although there is some general consensus. According to research, the fraction of particles that is small enough to stay suspended in air can be less than 0.1% (Mark Nicas, William W.N, Alan Hubbard, 2005)

Onto the environment variables, f_{at} is another composite variable and it accounts for the portion of the aerosolized droplets that go near the susceptible person. This variable is relevant as it can be used to model the impact of air currents, humidity, and temperature. (Mittal et al, 2020) For example, if an agent goes into close proximity with a window, and the program stipulates that he is impacted by an air current, then f_{at} will be adjusted accordingly. f_{at} can be expressed as the dividend of $\frac{CS}{CO}$. The variable CO is the aerosol concentration near the host and can be expressed as $\frac{R_h}{V_h}$ which are both variables in the host section of the inequality. The variable CS is once again a compound variable which represents the 'mean concentration of volume', VS at a distance of DHS and this is calculated to be approximately 2 liters at 1.5 meters of altitude.

Hence:

$$f_{at} = \frac{CS}{CO} = \frac{\frac{VS}{DHS}}{\frac{Rh}{Vh}} = \frac{\frac{2}{1.5}}{\frac{Rh}{Vh}}$$

The variable f_{vv} ‘represents the fraction of aerosolized droplets transported to the vicinity of the susceptible that contain viable virions.’ (Mittal et al, 2020) The authors explain that f_{vv} can be expressed as $e^{\frac{-T}{t}}$ where ‘ T is the time-of-flight and t is the half-life of virions in aerosolized form.’ The authors did not provide additional data, but research indicates that the half-life of COVID-19 seems to be 1.1 hours which would translate to 3960 seconds. (Ehsanifar, M.,2021). Research also shows that the predominance is that aerosols would stay suspended for 0.5h or 1800 seconds under normal circumstances. Mandavilli, A. (2020).

Hence an estimate for f_{vv} under normal circumstances could be:

$$f_{vv} = e^{\frac{-T}{t}} = e^{\frac{-3960}{1800}}$$

Onto the susceptible depend variables, f_{is} ‘represents the fraction of aerosols in the vicinity of the susceptible that would be inhaled by a susceptible not wearing a face covering’. (Mittal et al, 2020) The value of this variable in the context of an ABM simulation is that it allows us to account for discrepancies in how different people breathe, e.g. if an agent is a man, woman, child, or even if an agent suffers from a disability. It allows us to break the assumption that all agents breathe the same and it is a simple yet effective alternative to something more computationally intensive such as a fluid dynamics simulation. Research indicates that an adult human will inhale approximately 100 milliliters of air per second, and that this value could increase up to twenty times when performing physical activity, hence forth it was estimated in the CAT inequality study that the maximum amount an adult human can inhale will be about 2 liters of air per second. By analysing the provided data, it can be deduced that the standard value for f_{is} for an adult male ranged from 0.5-1 and approximately 20% less for women. Pre-teens can be expected to have an f_{is} of around 0.166 to 0.330. (Edwards, D.A, et al 2004 as cited in Mittal, et al 2020)

The variable f_{ms} is the equivalent of variable f_{mh} , but relates to the mask efficacy of the susceptible instead of the host.

The final variable is T_s , and it is a measure of time in seconds. Specifically, it measures how long the susceptible individual enters in contact with the aerosols that are suspended in the air from the host. This value is particularly useful as it is an incorporated counter which measures the time of contact between two agents.

The final variable is N_{ID} , which belong to the susceptible. It is the minimum dose of virions required to infect the individual, and will vary depending on the disease and the persons immune system. There are several studies available which provide possible values for this variable. In the case of Influenza, the minimum infectious dose is expected to be between 10 and 100 viral particles, and RSV between 10 to 1000 particles. (Beleser, J.A. et al, 2013) (Hall C.B., Douglas R.G. Jr., 1981) In the case of COVID-19 there is very limited information suggesting that it could range between around 500 viral particles to several thousands. (National Academies of Sciences 2020). Further, in the CAT inequality literature, there was no specified numerical format of how to apply the value of N_{ID} , meaning more information on is necessary. Although this may limit the reliability of the results achieved in the ABM simulation, it does not necessarily invalidate the use of the CAT framework for generating transmissions between agents. ‘The determination of N_{ID} for SARS-CoV-2 remains one of the most important tasks for scientists working in the arena’ (Mittal et al, 2020)

2.5 Agent-based Modelling Software

The modelling software is essential when developing ABM simulations and should be pondered thoroughly, as different tools have different advantages and disadvantages. Several software tools are available for developing ABM simulations. NetLogo, Python, Repast, Swarm, and Mason were studied and considered for our needs. Python was thoroughly experimented with; however, NetLogo was the final choice. Each tool has its own strengths and weaknesses, which may make it more suitable for specific tasks than others.

2.5.1. NetLogo and BehaviourSpace

NetLogo is a popular tool for developing agent-based models, especially in the social sciences. NetLogo was developed by Northwestern University’s Center for Connected Learning and Computer-Based Modelling, and it is free and open-source, which is an advantage for user

accessibility. Northwestern maintains the software and it works efficiently on the most common platforms.

The software has been used extensively in published scientific research papers, making it well respected in academia as a modelling software. Especially when dealing with predictions and simulations in medicine and biology, one must tread as carefully as possible when making inferences and conclusions on models. One wise provision to take is to use a software platform to build those models that is well known to the audiences of academia, where results can be easily scrutinized, dissected and compared to other works. This approach lessens the probability of surprise bugs appearing when dealing with peculiarities of custom-built systems, which are undesired. In a review article published by researchers at University of Catania, the use of NetLogo is praised for Agent-based modelling of the immune system and was found

‘to accurately reproduce the dynamics of TLR-4 signaling in response to LPS stimulation. In particular, it was capable of showing that there was a dose dependent proinflammatory response effect and also the establishment of tolerance.’ (Giallongo, et al, 2019)

The authors also reported on applications of NetLogo for the modelling of immunity to pathogens, modelling immune system dynamics, and tumour immunology. Hand in hand with immunology is virology, which is the topic of this very research paper. At University College Dublin, in 2018, researchers loaded real-world data into NetLogo to analyse factors in the spread of a measles-like disease' which caused an outbreak in Ireland in 2012. (Hunter, E., Mac Namee, B., Kelleher, J. 2018) The authors found that there was no correlation between the properties of specific towns and outbreaks. The authors also reported on applications of NetLogo for modelling immunity to pathogens, immune system dynamics, and tumour immunology. Given NetLogo's extensive use in research in biology and virology, it is an appropriate tool and the right choice for this project.

NetLogo also has the advantage of a fast learning curve due to its design tailored to scientists in fields outside of computer science. Despite considerable programming still involved in making a NetLogo situation, many issues programmers face are eliminated by using the software.

Due to the high-level language used to code models, called 'Logo', it has a lower opportunity cost than using other languages such as C++, Java and Python. Nevertheless, simulations in NetLogo can handle tens of thousands of agents, and while doing testing, there were no issues

with running the hospital simulation. Another significant advantage of using NetLogo is the fact that it has an integrated user-friendly GUI specifically tailored to agent simulations. Notably, the interface allows the user to interact with the simulation at the same time it is being executed and, thereby, change specific parameters on the go. It has different buttons, sliders, and switches in monitors, allowing for parameter adjustment and output visualization. It also has a built-in library of models that can be used as templates.

BehaviourSpace is a tool in NetLogo which allows for parameter sweeping and efficiently collecting information across several experiment runs. Although some experiments can take some time to run, the software was found to be generally fast for the purposes of this project.

2.5.2 Python with Pygame, Pandas and Matplotlib Libraries

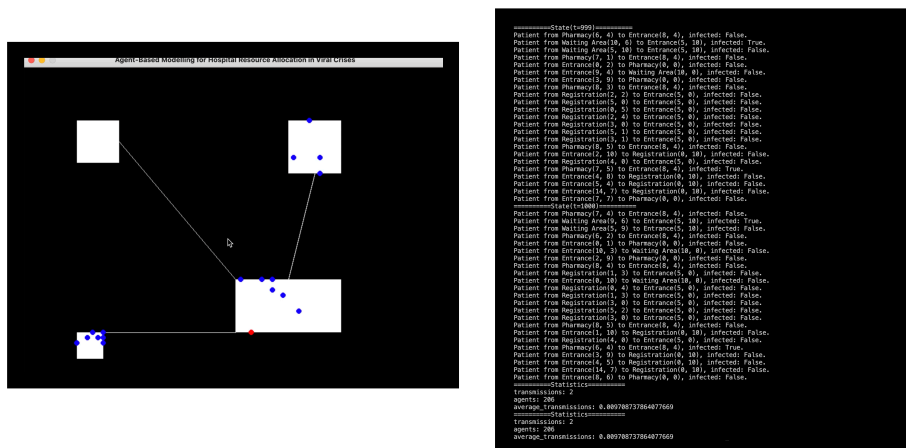


Figure 2: Initial prototype version of the desired system using Python and Libraries.

Python is a versatile and powerful programming language widely used in computer science, including in agent-based modelling. Python's vast libraries make handling data and complex simulations easier. It is also highly customizable, making it a good choice for more advanced users who want to create more complex simulations. However, Python requires more programming skills and may be more challenging for beginners. It will also require more effort to set up and customize than other tools. Python for ABM highlights creating a system from the ground up with a programming language and no modelling software framework. Using another language, such as Java or C++, would yield similar advantages and disadvantages, and the two programming languages are also used extensively in the industry for ABM.

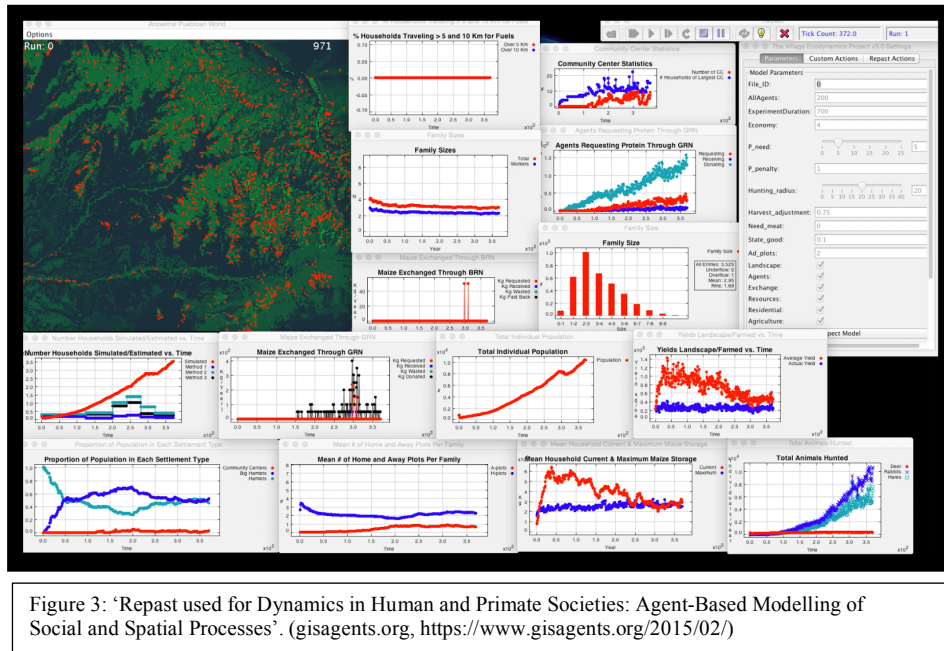
One of the main challenges in ABM is defining the behaviour of the agents. Each agent has unique properties and behaviours that must be incorporated into the model. For example, human agents may have different levels of susceptibility to the virus, and their behaviours may vary depending on the context, such as wearing masks or social distancing. Modelling such complex behaviours can be challenging and requires a deep understanding of the virus transmission dynamics and the Python programming language. In Python, even when taking advantage of several valuable libraries, planning and programming approximately 600 lines of code was necessary to set up an elementary sample environment with moving agents and a rudimentary infection model. The author was also not satisfied with how much coding was necessary to develop new appropriate, complex 2D environments. In indoor spaces, the building's layout, size, and ventilation systems can significantly impact the transmission of the

virus. These factors need to be accurately represented in the model for the simulation results to be meaningful. Python has several libraries that can be used for modelling the physical environment, such as Pygame and PyOpenGL. However, integrating these libraries with the virus transmission model can be time-consuming and requires 3D modelling and visualization expertise. NetLogo's built-in features, which allow for easy environment creation based on colour-coded rooms as image formats, were a significant advantage. The efforts to generate a friendly user interface and agent functionalities outweighed the freedom of control. It is only suggested that teams of multiple developers with various skills code such a system from scratch without using a modelling environment. Nevertheless, a well-built custom system can have near unlimited room for growth and bring new features that cannot be implemented in the modelling frameworks. It would be hoped that a running system used by hospital management where the outputs influence the safety measures and resource allocation be built from the ground up by dedicated teams.

2.5.3 The Recursive Porous Agent Simulation Toolkit (Repast)

Repast is another well-known toolkit software for developing agent-based models. The software uses an architecture based on Java, allowing for the use and integration of additional software libraries. However, Repast requires more programming skills than NetLogo and requires the user to learn how to use Java Groovy, making it less suitable for beginners.

Repast has a steeper learning curve and may require more effort to set up and customize. Although it is more customizable, the methods employed in this work are just as well commissioned in NetLogo, with the added benefit of a very easy-to-use and friendly interface for the final user.



2.5.4 Swarm

Swarm was, for a long time the go-to software for Agent-based modelling for many, which prompted the analysis of whether it should be used for our work. It was developed by the Santa Fe Institute, which specializes specifically in complex systems. One of the main reasons to avoid Swarm is that it is a relatively old platform, and its development has slowed down considerably, resulting in outdated modelling and visualization capabilities. The optimization techniques need to be updated, making it more challenging to run large-scale simulations within a reasonable time frame, which is essential for this project.

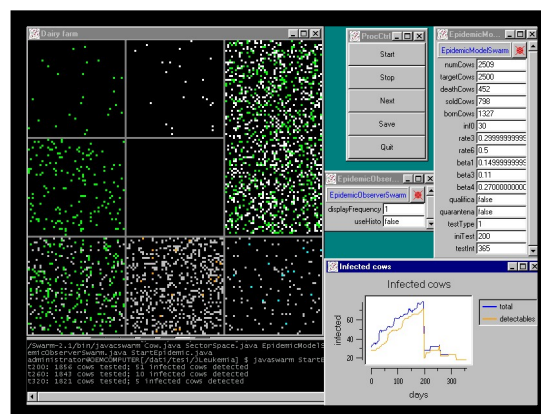


Figure 4: Swarm GUI
Bagni, R. ; Berchi. "A Comparison of Simulation Models Applied to Epidemics." Jasss.soc.surrey.ac.uk, 30 June 2002, jasss.soc.surrey.ac.uk/5/3/5.html.

Another reason to avoid Swarm is that it may not be as user-friendly as some of the available newer ABM platforms. The GUI is also not the friendliest for the end user to use. Swarm is written in Objective-C, which can make it difficult for those unfamiliar with this programming language to use. Additionally, the interface may need to be more intuitive than some modern platforms, making creating and modifying models more challenging. This means platforms such as Repast and NetLogo have surpassed it.

2.6 Patient Flow in Emergency Departments

This section covers the literature taken into consideration in order to represent the environment of a hospital emergency department in an ABM simulation.

2.6.1 Stavanger University Hospital Case Study

This dissertation heavily relies on research carried out by scientists at Stavanger University (Gaute Terning, Eric Christian Brun, Idriss El-Thalji, 2022) who modelled the flow of patients in an Emergency Department during COVID-19 pandemic conditions. Their research was invaluable and greatly supported the implementation of an environment where agents move around. In this dissertation, the primary elements used in the study were

- the blueprints of the hospital,
- the knowledge of the flow of patients
- the evaluation techniques used to determine the efficiency of resource allocation.

The study's authors used a hybrid approach of Agent-based and Discrete event modelling. Discrete event modelling focuses on the flow of events through a system, representing the system as a set of interacting components triggered by discrete events. The components of the system typically include people and other resources, and the events that trigger their interactions may include the arrival of patients, completion of tasks such as waiting in a room, or other important milestones. Discrete event modelling is more suited to discrete events and resource usage systems. In contrast, agent-based modelling is often used to represent complex systems that involve heterogeneous agents with varying behaviours and decision-making processes. (Robert Maidstone, 2012) Applying the outlined principles of patient flow by the authors of the Stavanger University Hospital study to the scope of this dissertation, it must be

conceded that elements of Discrete event modelling simulation are added to our ABM, although not enough to constitute and deem it a hybrid model.

The research authors explain how the Stavanger University Hospital managed patient inflows. The first stage begins with the patient's arrival at the emergency room. It ends when they have completed registration and are ready to be admitted into the triage area or treatment room. This stage takes place in the yellow area illustrated in the figure above. The entrance is indicated as a green area. All new patients arrived in the waiting room before the pandemic; however, during the pandemic, those who were positively identified as having the virus were inspected outside the door and taken directly to the treatment area, avoiding contact with other patients. The second stage is triage which consists of a patient waiting room and is denoted by the pink area on the map. The patients only go to triage to wait if no consultation rooms are available right away. The third stage is where medical care is provided. There are 11 bedrooms and 13 Blue treatment rooms. Those suspected of being infected would be rushed to a treatment room during the pandemic to reduce the possibility of transmission between patients. Before welcoming a new patient, the staff would clean the room after treatment. A patient in the treatment room for at least an hour must leave if the usual treatment is complete and a new patient with suspected contamination arrives. This is necessary to ensure that the contaminated patient does not remain with other patients. The fourth stage is the patient's discharge, which occurs when the treatment is over.

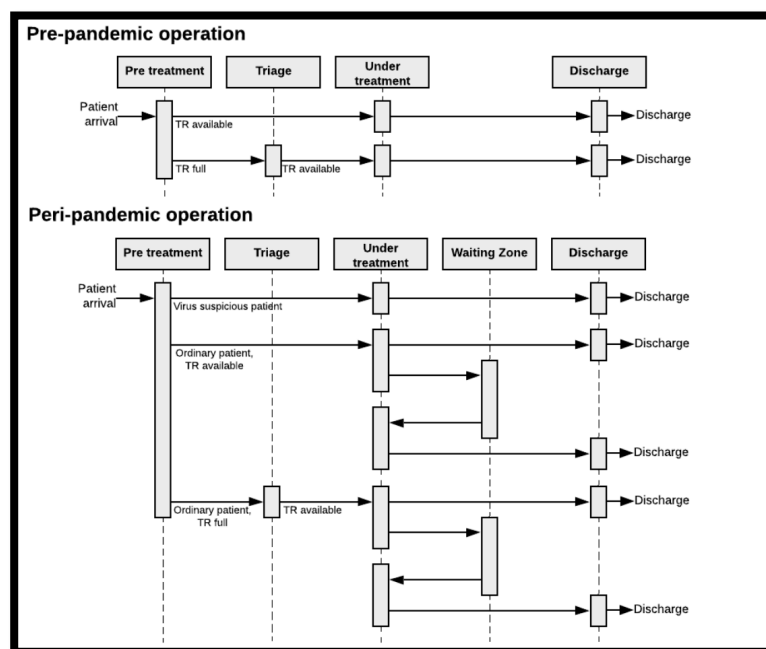


Figure 5: Patient Flow Diagram for Stavanger University Hospital (Gaute Terning, Eric Christian Brun, Idriss El-Thalji, 2022)

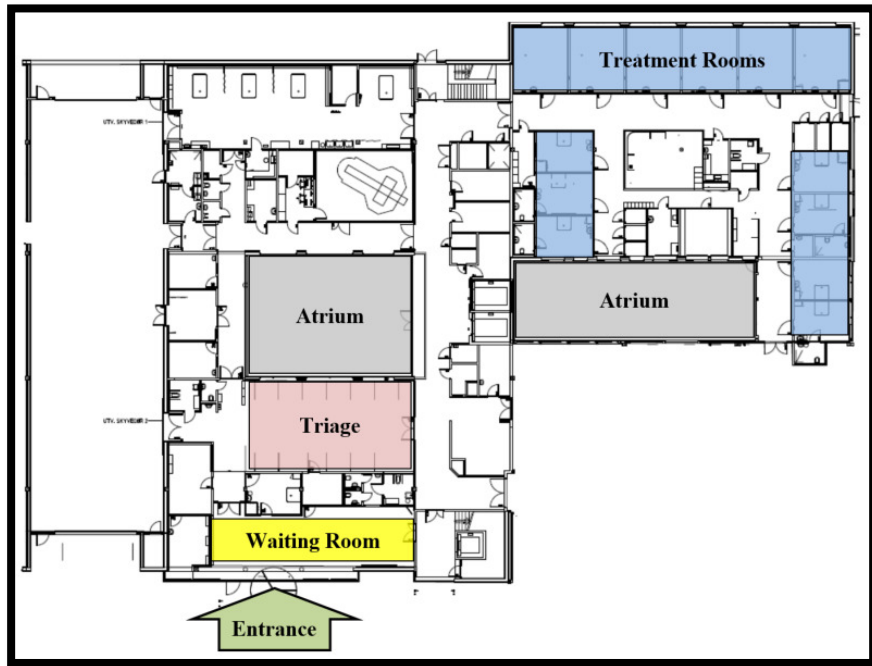


Figure 5: Stavanger University Hospital ER (Gaute Terning, Eric Christian Brun, Idriss El-Thalji, 2022)

Chapter 3

Requirements

This section outlines the fundamental requirements needed in the ABM simulation.

3.1 Environment Requirements

1. The environment should be visualized in NetLogo as an accurately scaled blueprint of the Stavanger Hospital emergency department.
2. The environment should display agents and show them moving around with the objective of following the patient flow process and acquiring treatment.
3. The aerosols deposited by infected agents should be shown in the map.
4. The user should be able to alter the environment if he so wishes, with his own blueprint.
5. The user should be able to control the following factors in the hospital environment:
 - a. The average time spent in the hospital reception (stage 1 of patient flow)
 - b. The average time that a patient spends in a doctor's appointment.
 - c. The average time that a hospitalized patient stays in the emergency room.
 - d. Define a populace amount where the hospital is considered to be crowded.

3.2 CAT Inequality Environment Requirements

1. The user should have control over the values of the environment variables defined in the CAT Inequality:
 - a. D_{hs}
 - b. F_{VV}
 - c. F_{at}

3.3 CAT Inequality Agent Requirements

1. The user should have control over the values of the Host variables defined in the CAT inequality. When an agent with the virus enters the simulation, he carries the following variables in his properties.
 - a. R_h
 - b. f_{vh}
 - c. f_{mh}
 - d. f_{ah}
2. The user should have control over the values of the Susceptible variables defined in the CAT inequality. When an agent without the virus enters the simulation, he carries the following variables in his properties.
 - a. f_{is}
 - b. f_{ms}
 - c. N_{ID}

3.4 Additional Agent Requirements

1. The user should have control over the anxiety level of Agents in the emergency department. The anxiety level is a measure of how much the agents move around within the hospital environment. In a perfect world agent would stay still, but in real life people move around while waiting to go to the bathroom or get a snack.
2. The user should have control over the probability that an agent that is infected, a host, arrived at the emergency department
3. The user should have control over the likelihood that an agent arrives at the hospital.

3.5 Graphical User Interface (GUI) Requirements

1. The GUI should be clear and user friendly
2. The user should be able to easily set-up and run the simulation with minimal to no coding.
3. The user should be able to see a series of informative statistics about the current state of the hospital during the given simulation run including:
 - a. The total number of patients

- b. The total number of infections that have occurred. Infections are defined as a host agent passing on the virus to susceptible agent.
- c. The total number of visitors the emergency department has received since the start of the run.
- d. The number of patients that are currently waiting in line to be attended by the reception
- e. The total number of infections directly related to waiting in line at the reception.
- f. The count of number of times a waiting room is seized by a patient.
- g. The number of rooms currently in use.
- h. How many patients are in the restroom, café or sitting in a chair
- i. How many patients are in the consulting room
- j. How many patients are bedridden.
- k. The total time to treatment, meaning the amount of time a patient has to be in the emergency room before he has access to the actual doctor.
- l. The average length of the stay of patients in the emergency department.
- m. The peak crowding time of the current run.
- n. How many times the emergency room becomes crowded.
- o. When the majority of patients are infected for the first time.

Chapter 4

Design, Implementation & Testing

4.1 – Introduction

In this chapter, we discuss the design process for the software system alongside the Implementation of the Design and how it was Tested.

4.2 – Design Process

The application passed through several design stages until it matured into its final version. The software engineering approach taken to develop and design the software was Iterative and Incremental Development. The reason for choosing this approach is that it allows for constant evaluation and room for change. Simulations are complex and error prone, hence a lot of testing is necessary and it goes hand-in-hand with the design and implementation—hence why it is also concatenated into chapter 4.

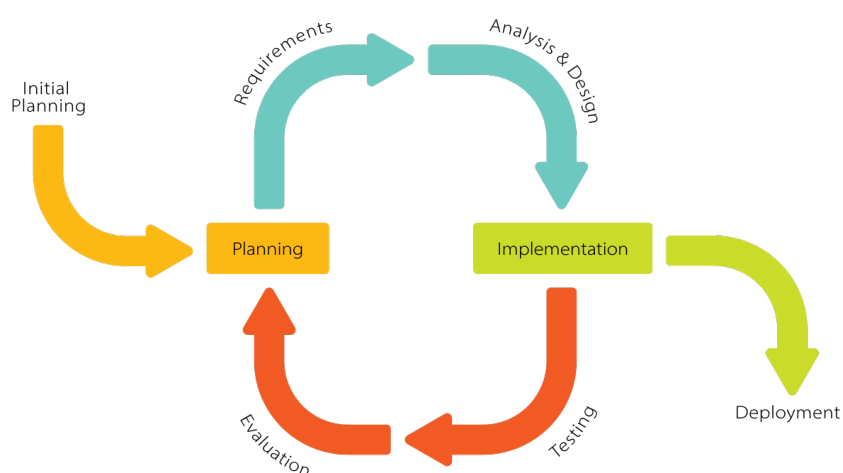
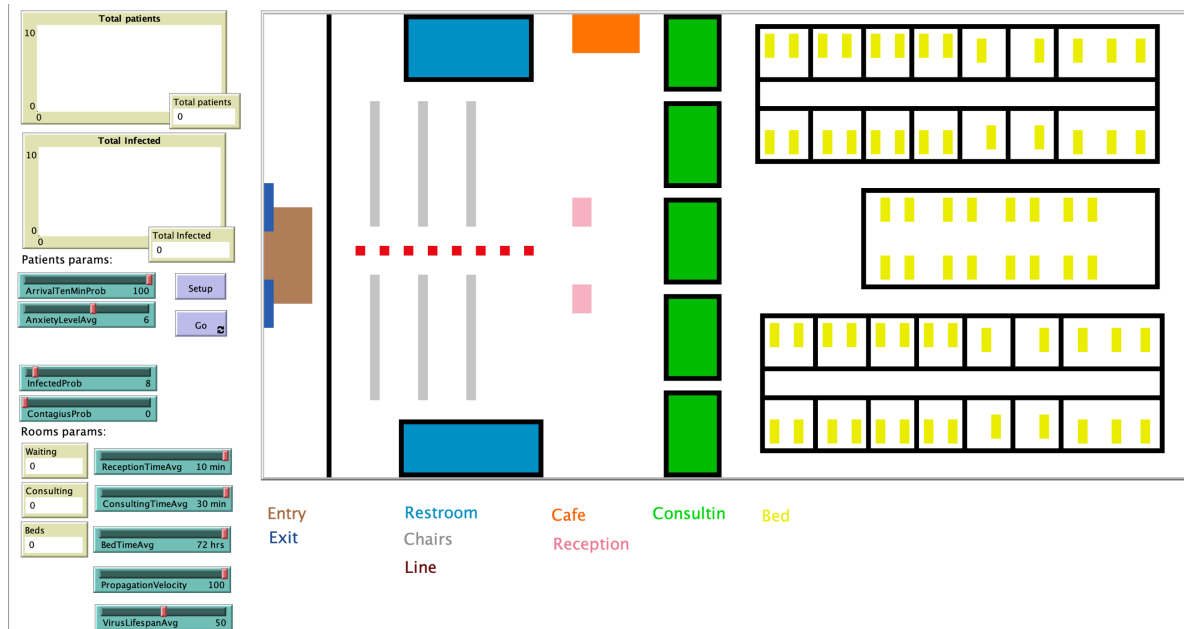


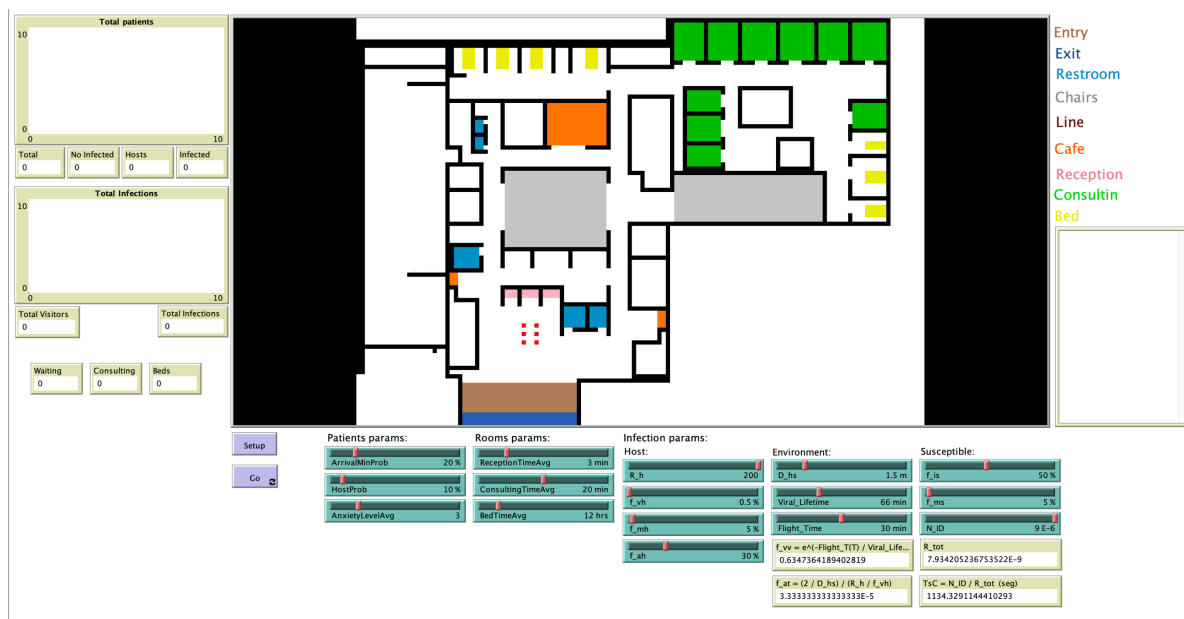
Figure 6: Krupadeluxe (2021)

Version 1:

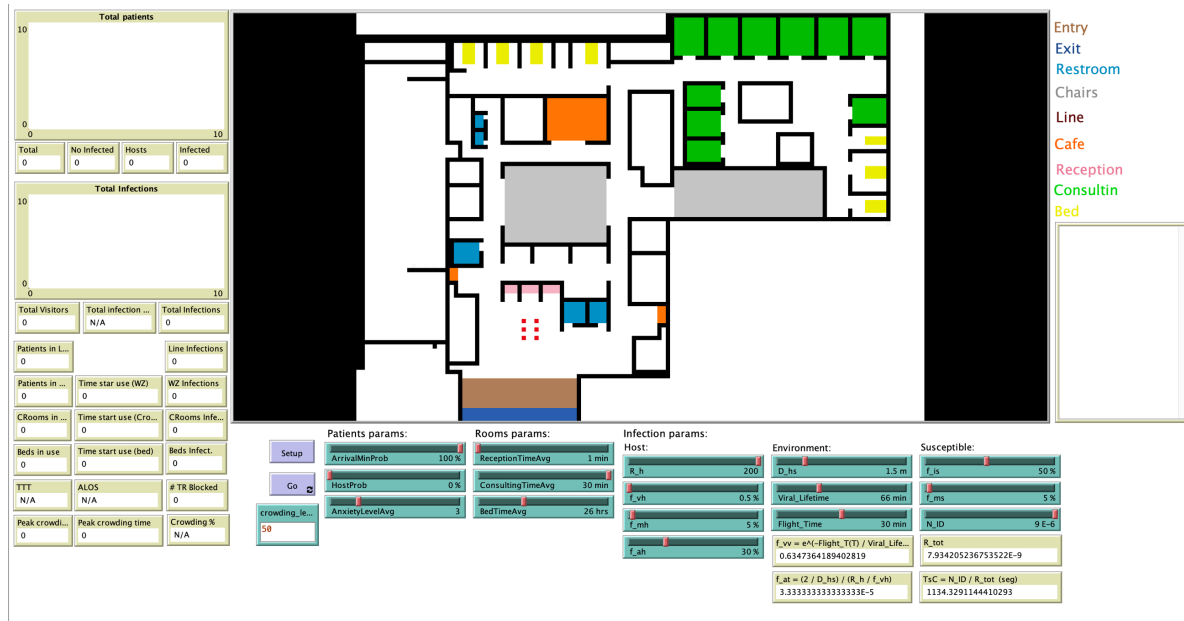


The initial version of the simulation in NetLogo looked like this. The map is an imaginary Emergency Department. At this point, there was a lot of testing regarding the movement model of the experiment. The CAT inequality was yet to be added, and patients would simply infect others based on probability.

Version 2:



The second version of the simulation implemented the CAT inequality parameters and the map layout based on the Stavanger University ED was implemented. Testing was done and the requirements for applying the CAT model were met. The simulation ran smoothly and all parameters worked as they should.

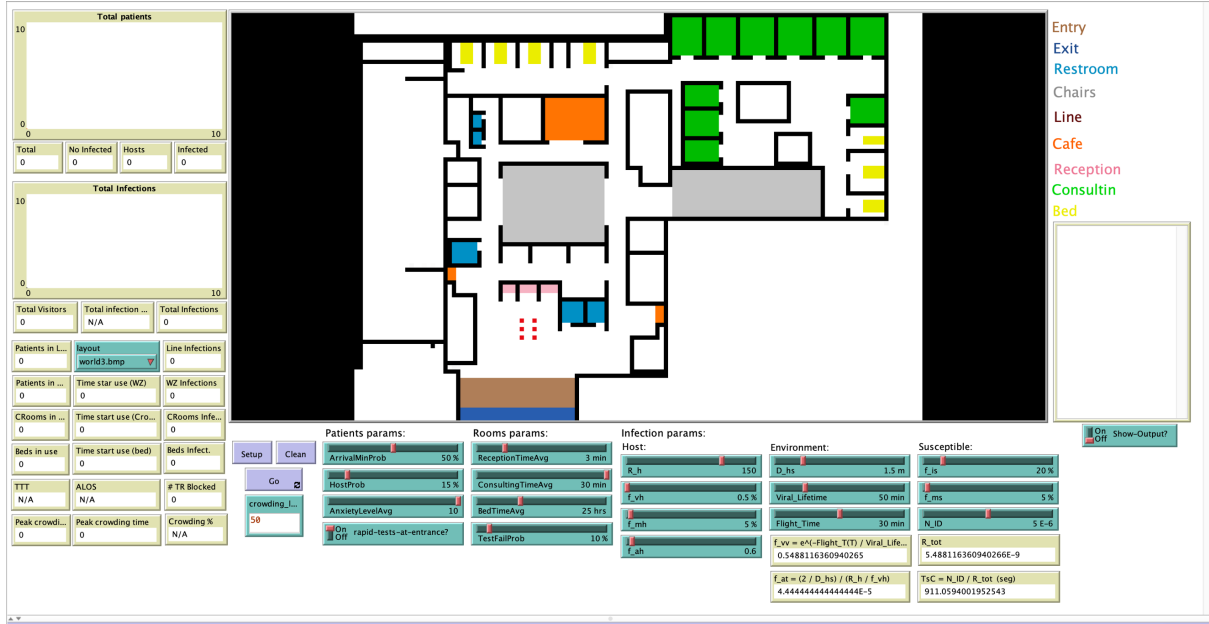


The third version of the simulation implemented all the relevant statistical outputs that were laid out in the requirements:

1. Total Visitors
2. Number of people infected in the Emergency Department
3. Number of Hosts in the Emergency Department
4. Number of patients waiting in line
5. Number of patients in the waiting zone
6. Number of infections that occurred in the waiting zone
7. Number of doctors' offices currently in-use
8. Number of beds for hospitalizations currently in use
9. Number of infected patients in hospital beds
10. Total Time to Treatment average (how long until patients meet the doctor)
11. Average length of stay in the hospital (how long until patients leave)

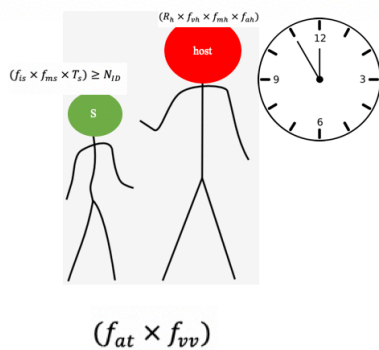
12. Number of consultations interrupted for susceptible patients to allow infected patients to have priority.
13. Live crowding data
14. The peak crowding time in the simulation world

Version 3: Final Version



The final version of the simulation added the ability to implement a measure of fast-tracking infected patients straight to the doctor's office. When the *rapid – tests – at – entrance?* Variable is set to true, rapid tests with a programmed accuracy are done before patients even enter the ED in order to detect hosts early on.

Illustrated Explanation of Transmission Model



If the two agents meet, or if the Susceptible enters the area with aerosolized particles recently emitted by the host, then a timer starts, T_s , and the variables of each agent are multiplied together and then multiplied by the environment variables. T_s increases linearly, which is what increases the value of the left-hand side of the Cat inequality. Once the left-hand side of the CAT inequality is equal or larger than the right, the susceptible agent is infected.

4.3 – Simulation Set-up

```
extensions [table]

breed [ patients patient ]

patches-own [ kind name occupied? originalColor virions decayRate cost ]

patients-own [
  status infectionStatus direction waitingTime triage
  Rh fvh fmh fah fis fms Rtot infected_count_in_WZ?
  EntryTime InfectionTime
]
```

In NetLogo, a breed is a set of agents that have the same set of variables and procedures. This section of code sets up the different breeds and variables used in the simulation. In the case of this simulation, we implemented only one breed called patients. This is because the primary agent we care about in the simulation is patients. In future work, hospital staff might also be an important breed to simulate, however, we are operating under the assumption that the hospital staff does not get sick to simplify the simulation, and therefore there is no need for the staff to actually be included.

The patches – own variable section sets up the following patch variables:

- *kind*: indicates the type of location the patch represents.
- *name*: used to identify the patch in the GUI.
- *occupied?* indicates if the patch is currently occupied by a patient.
- *originalColor*: stores the original color of the patch.
- *virions*: stores the number of virus particles on the patch.
- *decayRate*: stores the rate at which virus particles decay on the patch.
- *cost*: a table used to keep track of the cost of occupying a patch.

The *patients – own* variable section sets up the following patient variables:

- *status*: indicates the current status of the patient such as "arrive", "inDestination", "beingAttended" or "walking".
- *infectionStatus*: indicates if the patient is infected with the virus.
- *direction*: stores the current direction of the patient's movement.
- *waitingTime*: stores the amount of time the patient still has to wait.
- *triage*: stores the triage level assigned to the patient.

- $R_h, f_{vh}, f_{mh}, f_{ah}, f_{is}, f_{ms}, R_{tot}$: variables used to model the patient's immune response and susceptibility to the virus.
- *infected_count_in_WZ?*: indicates if the patient was infected while in the waiting zone.
- *EntryTime*: stores the time at which the patient entered the emergency room.
- *InfectionTime*: stores the time at which the patient was infected with the virus.

```

to setup
  let alreadySetup? any? patches with [kind = "consulting"]
  if (not alreadySetup?) or (alreadySetup? and user=yes-or-no? "Reload the map?" ) [
    setup-layout
  ]
  clean
end

to clean
  clear-globals
  clear-turtles
  clear-drawing
  clear-all-plots
  clear-output
  ask patches [
    set pcolor originalColor
    set virions 0
    set decayRate 0
    set occupied? false
  ]
  set messageFlag? false
  set virionSize (1 * (10 ^ -7))
  reset-ticks
end

```

This part of the code defines two procedures, *setup* and *clean*, which are used to initialize and reset a simulation in NetLogo. The *Setup* checks if any patches with kind equal to "consulting" have already been set up. If not, or if the user chooses to reload the map, the *setup-layout* procedure is called to create the patches and turtles for the simulation. Calls the *clean* procedure to reset the simulation.

The *clean* procedure clears all global variables, turtles, drawing commands, plots, and output. Iterates over all patches and sets their *pcolor* to *originalColor*, *virions* to 0, *decayRate* to 0, and *occupied?* to false. It sets the *messageFlag?* global variable to false. It sets the *virionSize* global variable to 1×10^{-7} . It resets the ticks to 0.

```

to setup-layout
  clear-patches
  clear-ticks
  no-display
  import-pcolors layout
  ask patches [
    set originalColor pcolor
    set name ""
    set kind ""
    set cost table:make
  ]
  ask patches with [pcolor <= 1][ set kind "border" ]
  ask patches with [pcolor >= 4 and pcolor <= 8][ set kind "chair" ]
  ask patches with [pcolor >= 14 and pcolor <= 18][ set kind "line" ]
  ask patches with [pcolor >= 24 and pcolor <= 28][ set kind "cafe" ]
  ask patches with [pcolor >= 34 and pcolor <= 38][ set kind "entry" ]
  ask patches with [pcolor >= 44 and pcolor <= 48][ set kind "bed" ]
  ask patches with [pcolor >= 64 and pcolor <= 68][ set kind "consulting" ]
  ask patches with [pcolor >= 94 and pcolor <= 98][ set kind "restroom" ]
  ask patches with [pcolor >= 104 and pcolor <= 108][ set kind "exit" ]
  ask patches with [pcolor >= 134 and pcolor <= 138][ set kind "reception" ]
  name-regions "chair"
  name-regions "line"
  name-regions "cafe"
  name-regions "entry"
  name-regions "bed"
  name-regions "consulting"
  name-regions "restroom"
  name-regions "exit"
  name-regions "reception"
  display
end

to name-regions [#kind]
  let n 0
  while [any? patches with [kind = #kind and name = ""]] [
    set n n + 1
    let newName word #kind n
    let pending-patches lput one-of patches with [kind = #kind and name = ""] []
    while [not empty? pending-patches] [
      let p first pending-patches
      set pending-patches but-first pending-patches
      ask p [
        set name newName
        set pending-patches lput neighbors with [kind = #kind and name = ""] pending-patches
        ask neighbors with [kind = #kind and name = ""] [
          set name newName
        ]
      ]
    ]
    ask one-of patches with [name = newName] [
      table:put cost name 0
      calculate-costs self name
    ]
  ]
end

```

This code defines the setup-layout procedure which is called by the setup procedure in the NetLogo modeling environment. The setup-layout procedure sets up the simulation world by creating the layout of the environment, with different regions of the environment marked by different colors. The first three lines of setup-layout clear the current world of patches and turtles, and disable the display to speed up the creation of the layout. Then, the import-pcolors command is used to import the layout from an external file specified by the variable layout.

After importing the layout, the ask patches command sets up the initial properties of the patches. The *originalColor* property is set to the current color of the patch, and the name and kind properties are set to empty strings. The cost property is set using the table:make command from the table extension. The following lines of code use the ask patches with [pcolor ...] command to identify patches with different colors and set their kind properties to match the regions they represent. The name-regions command is then called to assign unique names to each region. The display command is used to show the layout in the NetLogo interface.

4.4 – A* Pathfinding Algorithm

```
to calculate-costs [source-patch current_name]
  let current-patch 0
  let open []
  set open lput source-patch open
  let current-cost 0
  while [ length open != 0 ]
  [
    set current-patch item 0 open
    set open remove-item 0 open
    ask current-patch
    [
      set current-cost ( table:get [cost] of current-patch current_name ) + 1
      ask neighbors4 with [ kind != "border" and ((not table:has-key? cost current_name) or (table:get cost current_name) > current-cost) ]
      [
        set open lput self open
        table:put cost current_name current-cost
      ]
      set current-cost ( table:get [cost] of current-patch current_name ) + 1.4142
      ask neighbors with [ kind != "border" and ((not table:has-key? cost current_name) or (table:get cost current_name) > current-cost) ]
      [
        set open lput self open
        table:put cost current_name current-cost
      ]
    ]
  ]
end
```

This code implements an algorithm to calculate the cost of moving from one patch to another in a grid-based system. It uses the *A * algorithm* which is a popular pathfinding algorithm in computer science used to calculate the shortest path between the two patches. The code first initializes a list called "open" with the starting patch. It then enters a while loop that runs as long as there are items in the "open" list. In each iteration, the code gets the current patch by removing the first item from the "open" list. It then updates the cost of moving to each of the neighboring patches based on the cost of moving to the current patch plus the distance to the neighboring patch. It adds the neighboring patches to the "open" list if they haven't been visited before or if the cost of moving to them is less than the cost of moving to them previously. The algorithm continues until it has visited all patches that are reachable from the starting patch, and it records the cost of moving to each patch in a table called "cost".

4.5 – Main Loop

```
to go
  receive_patients
  if rapid-tests-at-entrance? [ test_patients ]
  move_patients
  contaminateSurface
  call_next_patient
  virionsDecay
  reTriage
  tick
end
```

This is the heart of the simulation, also known as the main loop. Here's a brief explanation of each of the procedures that are being called within it:

- *receive_patients*: checks if a new patient has arrived at the simulation and adds them to the patient queue if so.
- *if rapid – tests – at – entrance? [test_patients]*: Checks if the *rapid – tests – at – entrance?* global variable is true. If it is, calls the *test_patients* procedure to administer rapid tests to the patients.
- *move_patients*: for each patient in the hospital, determines their current location and destination, and then moves them towards that destination.
- *contaminateSurface*: calculates the amount of virus that is emitted by each patient and then spreads it to the environment, by updating the virions variable of the patches within a certain distance and angle from the patient.
- *call_next_patient*: checks if there are any available doctors or nurses and, if so, assigns them to the next patient in the queue.
- *virionsDecay*: simulates the decay of the virus over time. The virions variable of each patch is decreased based on the amount of time that has passed since the last go iteration and the decay rate of the virus.
- *Triage*: every 10 minutes, reduces the triage level of each patient by 1, simulating the gradual improvement or worsening of their condition.

- The tick increments the simulation clock by one.

4.6 – Rapid Tests at the Entrance

```
to test_patients
  ask patients with [ status = "arrive" and infectionStatus = "Host" ] [
    if (random 100 > TestFailProb) [
      set triage 0
      set label triage
    ]
  ]
end
```

This code defines a *test_patients* procedure that is used to administer rapid tests to patients in the simulation. This procedure simulates the administration of rapid tests to patients in the simulation environment, and updates their triage scores based on the results of the test. Patients who test positive will not have their triage score changed, and will therefore have lower priority for treatment.

ask patients with [status = "arrive" and infectionStatus = "Host"] selects all patients in the simulation environment that have just arrived and are currently Hosts

if (random 100 > TestFailProb) generates a random number between 0 and 100 and compares it to the *TestFailProb* global variable, which represents the probability of a test failing. If the random number is greater than the probability of a test failing, the test is considered successful and the patient's triage score is set to 0 (i.e., they have high priority for treatment). The label of the patient is also set to their triage score.

4.7 – Movement submodel

```

to receive_patients
  let myshapes ["man" "woman" "girl" "boy"]
  if (ticks = 0 or (ticks mod 10 = 0 and random 100 < ArrivalMinProb)) and not any? patches with [kind = "entry" and any? turtles-here] [
    carefully [
      ask one-of patches with [kind = "entry" and not any? turtles-here] [
        sprout-patients 1 [
          set total_visitors total_visitors + 1
          set size 3
          set status "arrive"
          set direction nobody
          set label-color black
          set triage -1
          set shape one-of myshapes
          set fis (calculateValue f_is 5 5 100) / 100 ; fraction of aerosols in the vicinity of the susceptible that would be inhaled by a sus
          set fms (calculateValue f_ms 5 0 100) / 100 ; fraction of inhaled aerosols that are NOT filtered
          set infectionStatus "No"
          if random 100 < HostProb [
            set infectionStatus "Host"
            set infected_count_in_WZ? false
            set Rh (calculateValue R_h 20 10 200) ; rate of expulsion of respiratory droplets ; change by gender / size
            set fvh (calculateValue f_vh 0.3 0.1 40) / 100 ; average number of virions contained in each expelled droplet
            set fmh (calculateValue f_mh 5 1 100) / 100 ; fraction of expelled droplets that make it past the face covering of the host
            set fah (calculateValue f_ah 0.2 0.1 (f_ah + 0.2)) ; fraction of expelled droplets that aerosolize
          ]
          (ifelse
            shape = "man" [ set color blue ]
            shape = "woman" [ set Rh Rh * 0.8 set fis fis * 0.8 set color pink ]
            shape = "boy" [ set Rh Rh * 0.6 set fis fis * 0.6 set color blue ]
            shape = "girl" [ set Rh Rh * 0.5 set fis fis * 0.5 set color pink ]
          )
          if infectionStatus = "Host" [set color red]
          if Show-Output? [
            output-type shape output-type "-" output-type who
            ifelse infectionStatus = "Host" [
              output-print (word " (Host):" )
              output-print (word "R_h: " precision Rh 4)
              output-print (word "f_vh: " precision fvh 4)
              output-print (word "f_mh: " precision fmh 4)
              output-print (word "f_ah: " precision fah 4)
            ] [
              output-print (word ":")
              output-print (word "f_is: " precision fis 4)
              output-print (word "f_ms: " precision fms 4)
            ]
            output-print ""
          ]
        ]
      ]
    ]
  ]
  let c count patients
  if peak_crowding < c [
    set peak_crowding c
    set peak_crowding_time ticks
  ]
  if c > crowding_level [
    set crowding_time_accumulated crowding_time_accumulated + 1
  ]
end

```

The *receive_patients* procedure is responsible for generating new patients and placing them in the simulation environment. It starts by defining a list of shapes to use for the patient agents. It then checks if it's time to generate new patients by checking if the current tick is a multiple of 10 and a random number is less than the *ArrivalMinProb*. If this condition is met and there are no other patients in the entry patches, it creates a new patient agent on one of the available entry patches using the *sprout-patients* command. Each new patient is given a set of attributes such as *triage*, and *infectionStatus*. These attributes help to keep track of the patient's state and behavior during the simulation. The patient's attributes are also set based on the patient's

shape and infection status. These attributes are used to calculate the probability of infection, transmission, and other related parameters. *If the Show – Output?* flag is set to true, the procedure outputs the shape and infection status of each new patient and their related attributes. Finally, the procedure checks if the current number of patients in the simulation exceeds a *crowding_level* threshold. If so, it increments the *crowding_time_accumulated* variable, which keeps track of the total time the number of patients has exceeded the threshold. The *peak_crowding* variable is also updated to keep track of the maximum number of patients in the simulation at any given time.

```
to move_patients
  ask patients [
    breath
    (ifelse
      status = "arrive" [ move_patients_arrive ]
      status = "walking" [ move_patients_walking ]
      status = "inDestination" [ receive_patients_inDestination ]
      status = "beingAttended" [ attended_patients ]
    )
  ]
end
```

This function acts as a “switch” that moves patients depending on their current state. Moving does not necessarily mean walking, this can also mean continuing to be cared for, continuing to wait, etc. Also, in this procedure is when patients breathe.

```

to call_next_patient
  let nextPatient one-of patients with [trriage = 0 and infectionStatus = "Host"]
  if nextPatient = nobody [ set nextPatient min-one-of patients with [trriage != -1 ] [trriage] ]
  if any? patches with [kind = "consulting" and occupied? = false] and nextPatient != nobody [
    let p one-of patches with [kind = "consulting" and occupied? = false]
    ask nextPatient [
      set triage -1
      set label ""
      leavePlaceAndGoTo p
      set total_time_to_be_attended total_time_to_be_attended + (ticks - EntryTime)
      set total_patients_attended total_patients_attended + 1
    ]
  ]
end

```

The *call_next_patient* function is used to determine the next patient to be attended and assigns them to an unoccupied consulting patch. The method first looks for a patient with a triage score of 0 and an infection status of "Host", which means they are the source of the infection. If it can't find such a patient, it selects the patient with the lowest triage score that hasn't been attended yet. If there is an unoccupied consulting patch and a patient is selected, the patient is assigned to the patch, their triage score is set to -1, and their label is cleared. Then, the patient leaves their current place and goes to the consulting patch. Finally, some statistics are updated regarding the total time patients have waited to be attended and the total number of patients attended.

```

to move_patients_arrive
  let p nobody
  (ifelse
    any? patches with [kind = "line" and occupied? = true] [
      set p min-one-of patches with [ kind = "line" and occupied? = false ] [distance one-of patches with [kind = "reception"]]
    ]
    any? patches with [kind = "reception" and occupied? = false] [
      set p one-of patches with [kind = "reception" and occupied? = false]
    ]
    any? patches with [kind = "line" and occupied? = false] [
      set p min-one-of patches with [ kind = "line" and occupied? = false ] [distance one-of patches with [kind = "reception"]]
    ]
  )
  [
    move_nextZone ["chair" "cafe" "restroom"]
  ]
)
if p != nobody [ set EntryTime ticks ]
leavePlaceAndGoTo p
end

```

This procedure *move_patients_arrive* is called when patients are in the "arrive" status. It attempts to move the patient to the next zone, which could be a line, the reception, or any of the areas specified in the *move_nextZone* procedure. If there is at least one line that is occupied by another patient, then the procedure will attempt to find a line that is not occupied and closest to the reception area. This simulates the real life behaviour of people and how they

are expected to act. If there are no lines that are unoccupied, then the method will look for an unoccupied patch in the reception area. If there are no unoccupied patches in the reception area, then the procedure will look for an unoccupied line that is closest to the reception area. If none of these conditions are satisfied, the *move_nextZone* procedure is called to move the patient to another zone. Finally, the *leavePlaceAndGoTo* procedure is called to move the patient to the destination patch. If a destination patch was found, the *EntryTime* variable is set to the current tick.

```

to move_patients_walking
  let nextPatch direction
  ifelse [name] of direction = [name] of patch-here [
    if direction = patch-here [
      set status "inDestination"
      stop
    ]
  ][
    let current_name [name] of direction
    set nextPatch min-one-of neighbors with [ table:has-key? cost current_name ] [
      table:get cost current_name
    ]
    if nextPatch = nobody [
      if Show-Output? [ output-show (word "No route found!! ") ]
      die
      stop
    ]
  ]
  face nextPatch
  fd 1
end

```

The *move_patients_walking* procedure handles the movement of patients by foot throughout the hospital environment. The code is a simple pathfinding algorithm that moves patients towards their destination, using a cost table to determine the optimal path. The “let *nextPatch* direction” line assigns the *nextPatch* variable to the patch in the direction that the patient is facing. The ifelse statement checks whether the name of the direction patch is the same as the name of the current patch (i.e., the patient has reached their destination). If it is, the patient's status is set to "inDestination", and the procedure stops. If the patient has not yet reached their destination, the code enters the else block. The “let *current_name* [name] of direction” line assigns the name of the direction patch to the *current_name* variable. The “set nextPatch min-

one-of neighbors with [table:has-key? cost current_name] [table:get cost current_name]” line finds the neighboring patch with the lowest cost value for the current_name key in the cost table. This determines the direction that the patient should move in. If there is no valid path, the code displays an output message and stops the procedure. Finally, the patient is turned to face the nextPatch and moved forward by one unit using face nextPatch and fd 1, respectively.

```
to receive_patients_inDestination
  set status "beingAttended"
  let currentPlaceKind [kind] of direction
  (ifelse
    currentPlaceKind = "reception" [ set waitingTime calculateValue ReceptionTimeAvg 5 1 10 ]
    currentPlaceKind = "line" [ set waitingTime -1 ]
    currentPlaceKind = "chair" [
      calculateStatsinWZ
      set waitingTime 15 - calculateValue AnxietyLevelAvg 2 1 10
    ]
    currentPlaceKind = "cafe" [
      calculateStatsinWZ
      set waitingTime calculateValue 10 8 1 25
    ]
    currentPlaceKind = "restroom" [
      calculateStatsinWZ
      set waitingTime calculateValue 3 10 1 15
    ]
    currentPlaceKind = "consulting" [
      if start_time_consulting = 0 [ set start_time_consulting ticks ]
      set waitingTime calculateValue ConsultingTimeAvg 5 10 30
    ]
    currentPlaceKind = "bed" [
      if start_time_bed = 0 [ set start_time_bed ticks ]
      set waitingTime ( calculateValue BedTimeAvg 8 1 72 ) * 60
    ]
    currentPlaceKind = "exit" [ exit ]
  )
end
```

This part of the code handles the process of receiving patients at their destination. This piece of code is built to handle the waiting time calculations for patients at different locations in the simulation, based on the type of the location and other variables such as the patient's anxiety level or the start time of the patient's stay at the location. The waiting time is calculated based on certain averages and randomized values, and patients are removed from the simulation when they reach the exit area. The first line updates the patient's status to "beingAttended". The “let currentPlaceKind [kind] of direction” line assigns the *currentPlaceKind* variable to the kind of the patch in the direction the patient is facing. The ifelse statement checks the *currentPlaceKind* and sets the *waitingTime* accordingly: The given variable (In this case *ReceptionTimeAvg*) is the mean of a normal distribution. Therefore, it will be assigned a normally distributed value between 1 and 10 (min and max) with a given mean and a sigma

(interval), where the higher the sigma value, the more distributed that value will be between the range. (The bell will be flatter), while smaller, the values will be more concentrated in the mean. If the patient is in a line, *waitingTime* is set to -1, indicating that the patient does not have to wait in line. If the patient is in a chair, the *calculateStatsinWZ* function is called to calculate statistics related to the patient's wait time, and *waitingTime* is set to 15 minus the average anxiety level plus a random value between 1 and 10. If the patient is in a cafe, the *calculateStatsinWZ* function is called to calculate statistics related to the patient's wait time, and *waitingTime* is set to a random value between 8 and 25. If the patient is in a restroom, the *calculateStatsinWZ* function is called to calculate statistics related to the patient's wait time, and *waitingTime* is set to a random value between 1 and 15. If the patient is in a consulting area, *waitingTime* is set to the average consulting time plus a random value between 10 and 30. If this is the patient's first time at the consulting area, *start_time_consulting* is set to the current ticks value. If the patient is in a bed area, *waitingTime* is set to the average bed time multiplied by 60, indicating the duration of the patient's stay in the bed. If this is the patient's first time in the bed area, *start_time_bed* is set to the current ticks value. If the patient is at the exit area, the exit command is called to remove the patient from the simulation. The end keyword indicates the end of the *receive_patients_inDestination* procedure.

```

to attended_patients
  let currentPlaceKind [kind] of direction
  (ifelse
    waitingTime <= 0 [
      (ifelse
        currentPlaceKind = "reception" [ move_patients_fromReception ]
        currentPlaceKind = "chair" [ move_patients_InTheWaitingRoom ]
        currentPlaceKind = "cafe" [ move_patients_InTheWaitingRoom ]
        currentPlaceKind = "restroom" [ move_patients_InTheWaitingRoom ]
        currentPlaceKind = "consulting" [ move_patients_InConsulting ]
        currentPlaceKind = "bed" [ move_patients_InBed ]
      )
    ]
    waitingTime > 0 [
      if ticks mod 10 = 0 [set waitingTime waitingTime - 1]
    ]
  )
end

```

This *attended_patients* procedure is used to handle patients who have been attended to by a staff member in the Emergency Department simulation. The first line retrieves the kind attribute of the patch the patient is currently on. This is used to determine which type of location the patient is currently in. The ifelse statement checks if the patient's *waitingTime* is less than or equal to 0. If it is, this means that the patient has been attended to by a staff member and needs to move to a new location in the simulation. The nested ifelse statement checks the current location of the patient and calls the appropriate *move_patients_** procedure to move the patient to the next location. If the patient's *waitingTime* is greater than 0, the if statement checks if the current simulation time ticks is a multiple of 10. If it is, the patient's *waitingTime* is decremented by 1. The *attended_patients* procedure does not end with a stop or die statement, so it will continue to execute until the simulation ends or the patient is moved to a new location.

```

to move_patients_fromReception
  move_nextZone ["chair" "cafe" "restroom"]
  ifelse triage = 0 [
    move_nextZone ["exit"]
  ] [
    ifelse infectionStatus = "Host" [
      set triage floor calculateValue 1 1 0 5
    ] [
      set triage floor calculateValue 3 2 0 5
    ]
    set label triage
  ]
  let patientsInLine patients with [direction != nobody and [kind] of direction = "line"]
  let patientsInLineSorted sort-on [ distance myself ] patientsInLine
  foreach patientsInLineSorted [ [x] ->
    ask x [
      let p nobody
      ifelse any? patches with [kind = "reception" and occupied? = false] [
        set p one-of patches with [kind = "reception" and occupied? = false]
      ] [
        set p min-one-of patches with [ kind = "line" and occupied? = false ] [distance one-of patches with [kind = "reception"]]
      ]
      leavePlaceAndGoTo p
    ]
  ]
end

```

This code defines a procedure called *move_patients_fromReception* that moves patients from the reception area to other areas in the simulation. The procedure first calls another procedure *move_nextZone* with a list of zones to move to ("chair", "cafe", "restroom") in that order. Next, it checks the patient's triage status. If the patient is untriaged (triage = 0), they are moved to the "exit" zone. If the patient is triaged, they are assigned a new triage level based on their infection status, and their label is updated accordingly. The procedure then selects all patients in the "line" zone whose direction is not nobody. It sorts these patients by their distance from the current agent and iterates over them. For each patient, the agent attempts to find a free patch in the "reception" zone or, failing that, the "line" zone that is closest to a free patch in the "reception" zone. The patient then leaves their current patch and moves to the selected patch.

```

to move_nextZone [#options]
  vacate_zone
  let d one-of #options
  while [not any? patches with [kind = d and occupied? = false]] [
    set d one-of #options
  ]
  set direction one-of patches with [kind = d and occupied? = false]
end

to leavePlaceAndGoTo [#p]
  if #p != nobody [
    vacate_zone
    set direction #p
    ask patches with [name = [name] of #p] [set occupied? true]
  ]
end

to vacate_zone
  if direction != 0 and direction != nobody [
    let currentLocationName [name] of direction
    ask patches with [name = currentLocationName] [set occupied? false]
  ]
  set status "walking"
end

to reTriage
  if ticks mod 600 = 0 [
    ask patients with [triage > 0] [
      set triage triage - 1
      set label triage
    ]
  ]
end

```

The *move_nextZone* procedure selects a new direction for the patient to move in. It takes an input list *#options* that specifies the possible kinds of zones the patient can move to. It first calls the *vacate_zone* procedure to free up the current location of the patient. It then randomly selects one of the *#options* until it finds an unoccupied patch of the corresponding kind. Finally, it sets the patient's direction to the selected patch.

The *leavePlaceAndGoTo* procedure vacates the current location of the patient by calling the *vacate_zone* procedure and then sets the patient's direction to the input patch *#p*. It also marks the patch as occupied by the patient.

The *vacate_zone* procedure marks the current location of the patient as unoccupied by setting the *occupied?* attribute of the corresponding patch to false. It also sets the patient's status to "walking".

The *reTriage* procedure is called every 600 ticks (10 minutes) and decrements the triage value of patients who have a value greater than 0. It also updates the label of these patients to reflect their new triage value.

4.8 – Contagion sub-model

```

to breath
  let virionsHere [virions] of patch-here
  if virionsHere > 0 and direction != nobody [
    let virionsInhaled (virionsHere * fis * fms)
    let inhaled min (list virionsInhaled virionsHere R_tot)
    set virionsHere (virionsHere - inhaled)
    if virionsHere < virionSize [ set virionsHere 0 ]
    ask patch-here [
      set virions virionsHere
    ]
    set Rtot Rtot + inhaled
    if infectionStatus = "No" and Rtot > (N_ID * (10 ^ -6)) [
      set infectionStatus "Infected"
      set InfectionTime ticks
    ]
    (ifelse
      [kind] of direction = "line" or [kind] of direction = "reception" or [kind] of patch-here = "line" or [kind] of patch-here = "reception" [
        set infections_Line infections_Line + 1
      ]
      [kind] of direction = "restroom" or [kind] of direction = "cafe" or [kind] of direction = "chair" [
        set infections_WZ infections_WZ + 1
      ]
      [kind] of direction = "consulting" or [kind] of patch-here = "consulting" [
        set infections_CRooms infections_CRooms + 1
      ]
      [kind] of direction = "bed" or [kind] of patch-here = "bed" [
        set infections_Beds infections_Beds + 1
      ]
    )
    if Show-Output? [ print (word shape "-" who ": Infected after " ((InfectionTime - EntryTime) / 10) "min. (" ([kind] of patch-here) " " status " ")) ]
    set color orange
    set total_infections total_infections + 1
    if Show-Output? and (not messageFlag?) and count patients with [infectionStatus = "Infected"] > count patients with [infectionStatus = "No"] [
      user-message (word "At " precision (ticks / 600) 2 "hrs. the majority of the patients are infected for first time")
      set messageFlag? true
    ]
  ]
end

```

The function *breathing* is used to model a patient breathing and potentially inhaling virions. The first line defines a variable called "*virionsHere*" as the number of viral particles present in the patch location where the patient is currently located. The next line checks if there are any viral particles present in the patch, and if the patient has a direction to move towards. The following line calculates the number of viral particles that the patient inhales based on various factors. The "*inhaled*" variable is then set to the minimum value of the calculated viral particles, the number of viral particles present in the patch, and the constant R_{tot} . The number of viral particles present in the patch is then updated by subtracting the "*inhaled*" variable from it. If there are no more viral particles left in the patch, the "*virionsHere*" variable is set to 0. The "*virions*" variable of the patch where the patient is located is then updated with the new "*virionsHere*" value. The " R_{tot} " variable, which represents the total number of viral particles that the patient has inhaled so far, is updated by adding the "*inhaled*" variable to it. If the patient has not been infected yet and the " R_{tot} " variable exceeds a certain threshold, the

patient is marked as infected and the infection time is recorded. The code then checks the location of the patient and updates the infection count for that location accordingly. If the "Show – Output?" variable is set to true, the function prints a message indicating that the patient has been infected and for how long. The colour of the patient is also updated to orange to indicate infection. The total infection count is updated by incrementing it by one.

If the "Show – Output?" variable is true, the function checks whether a message indicating that the majority of patients are infected for the first time has already been displayed. If not, it displays the message and sets a flag to indicate that the message has been displayed.

```

to contaminateSurface
ask patients with [infectionStatus = "Host" and status != "arrive" ] [
  let angle 360
  if status = "walking" [
    set angle 180
  ]
  let expelledVirions (6 * Rh * fvh * fmh) ; 1 tick = 6 seg
  let aerosolizedVirions (expelledVirions * fah)
  let tempAerosolizedVirions aerosolizedVirions
  let SpreadRatio ( D_hs * 2.5 )
  let Co (Rh / fvh)
  ask patches in-cone SpreadRatio angle with [ kind != "border" and kind != "exit" and kind != "entry" ] [
    if tempAerosolizedVirions > 0 [
      let d (distance myself)
      if d > 0 [
        let f_at ( (2 / d) / Co )
        let v ( aerosolizedVirions * f_at * f_vv )
        set virions virions + v
        calculate_decayRate
        set tempAerosolizedVirions (tempAerosolizedVirions - v)
      ]
    ]
  ]
]
end

```

This is a function that calculates the spread of viral particles from infected patients to the surfaces in the environment. The function begins by asking all patients with an "infectionStatus" of "Host" and not in the "arrive" status to execute the code within the brackets. It then calculates the expelled and aerosolized viral particles based on a set of variables. The "expelledVirions" variable is determined by multiplying the variables " R_h ," " f_{vh} ," and " f_{mh} ." The "aerosolizedVirions" variable is determined by multiplying the "expelledVirions" variable by the " f_{ah} " variable, which represents the fraction of the virus aerosolized.

The function then calculates the spread ratio, "*SpreadRatio*" which is based on the distance between the patient and the surface being contaminated. It multiplies the "*SpreadRatio*" by 2.5, which is an arbitrary factor, to obtain the radius of the cone of contamination. The function also calculates the "*Co*" variable, by dividing "*R_h*" by "*f_{vh}*".

The function then uses the "ask" statement to iterate over all patches that are within the cone of contamination and have a "kind" that is not "border," "exit," or "entry." It checks if there are any aerosolized viral particles left to be spread, and if so, it calculates the distance "d" between the patient and the patch being contaminated. If the distance is greater than zero, it calculates the *f_{at}* between the viral particles and the patch, and the "v" of the viral particles that will settle on the patch. The function then sets the "*virions*" variable of the patch to the sum of its current value and the newly settled viral particles, and calculates the decay rate of the viral particles. Finally, the function subtracts the settled viral particles from the remaining aerosolized viral particles, and the iteration continues until there are no more aerosolized viral particles to be spread.

```

to-report calculateValue [avg interval mn mx]
  let escale interval / 3
  loop [
    let X random-normal 0 1
    let r avg + (X * escale)
    if r >= mn and r <= mx [ report r ]
  ]
end

to-report calculate_fvv
  set f_vv ( e ^ ( -1 * ( Flight_Time / Viral_Lifetime ) ) )
  report f_vv
end

to-report calculate_rtot
  set R_tot R_h * (f_vh / 100) * (f_mh / 100) * (f_ah / 100) * ((2 / D_hs) / (R_h / (f_vh / 100))) * f_vv * (f_is / 100) * (f_ms / 100)
  report R_tot
end

```

These are functions used to calculate various parameters related to the spread of the virus in the simulation.

The *calculateValue* function uses a normal distribution to generate random values around a given average value, within a specified interval and bounds. It can also be recognized as Sigma. We know that in the normal distribution 99% of the values will be at most 3 times sigma away from the mean.

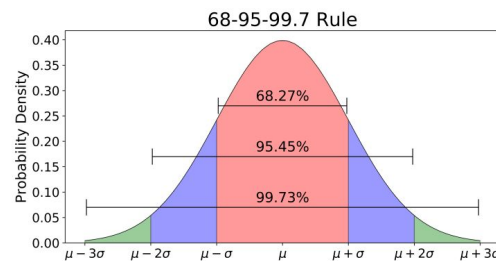


Figure 7: 68, 95, 99.7 Rule : Michael Galarnyk (2018)

The *calculate_fvv* function calculates the fraction of viable virus particles that remain in the air after a given amount of time, based on the flight time and viral lifetime of the virus.

The *calculate_rtot* function calculates the total number of virus particles that can potentially be transmitted in the environment, considering various factors such as the infectivity, susceptibility, and movement of individuals in the simulation.

```

to calculate_color
  if virions <= virionSize [
    set pcolor originalColor
    set virions 0
    stop
  ]
  let virionColor virions * 566
  if virionColor > 8.5 [ set virionColor 8.5 ]
  set pcolor ( 19.5 - virionColor )
end

to calculate_decayRate
  let k (( ln 2 ) / (Viral_Lifetime * 10)) ; in tiks
  let t ( ln (virionSize / virions) / (-1 * k) )
  set decayRate ( (virions - virionSize) / t )
  set decayRate max list decayRate virionSize
end

```

The *calculateValue* procedure takes three inputs *avg*, *interval*, *mn*, and *mx*, and returns a random value that falls within the range of *mn* and *mx* with a normal distribution centered around *avg*. The range of random values is controlled by *interval*.

The *calculate_fvv* procedure calculates the decay factor of the virus particles in the air over a certain time period, where *Flight_Time* is the time the virus particles remain suspended in the air, and *Viral_Lifetime* is the time it takes for the virus particles to lose half of their infectiousness.

The *calculate_rtot* procedure calculates the total transmission rate of the virus, based on several factors such as the infectivity of the virus in different environments, the transmission efficiency of the virus from host to host, and the concentration of virus particles in the air.

The *calculate_color* procedure determines the color of a patch based on the concentration of virus particles present in it. If the concentration is below a certain threshold (*virionSize*), the patch color remains unchanged. Otherwise, the concentration of virus particles is mapped to a color scale, and the patch color is set accordingly.

Finally, the *calculate_decayRate* procedure calculates the rate at which the virus particles lose their infectiousness over time, based on the initial concentration of virus particles and the *Viral_Lifetime* value. The rate of decay is set to zero if the virus concentration falls below a certain threshold (*virionSize*).

Chapter 5

Experimentation

The objective of this research is to determine how an ABM simulation, using the CAT inequality, can be useful to the management entities responsible for the functioning of emergency departments. In this section, a series of experiments are run using BehaviourSpace in NetLogo. Please refer to Appendix A which has all the data generated by BehaviourSpace for the experiments.

5.1 Experimenting with mask parameters

The use of masks was one of the most discussed topics during the COVID-19 pandemic, hence a series of tests were carried out in BehaviourSpace to determine the impacts of these face coverings on the Emergency Department.

Total simulations: 54

BehaviourSpace Parameters:

HostProb = [6]

AnxietyLevelAvg = [5]

F_mh = [5, 10, 100]

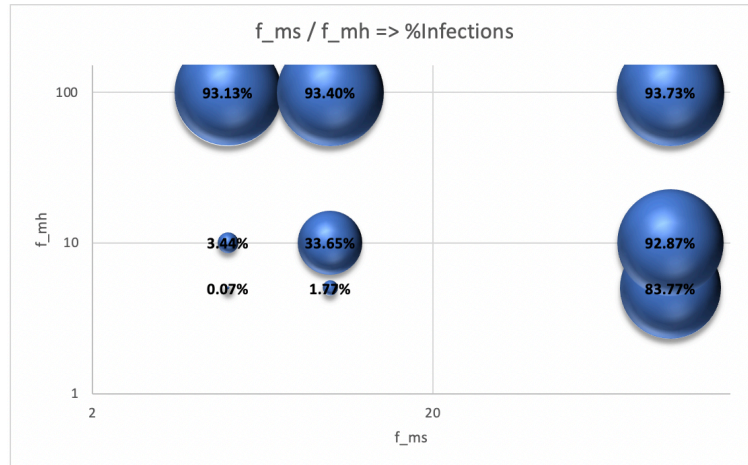
F_ms = [5, 10, 100]

RecpetionTimeAvg = [3]

ConsultingTimeAvg = [20]

Rapid-test-at-entrance = [false, true]

Figure 8:



The results of the experiment clearly show that as f_{mh} and f_{ms} increase, the percentage of infections also increases greatly. This is because the higher the value of f_{mh} and f_{ms} the less effective masks are. The table containing the full experiment results in the Appendix section A shows that the non-use of effective masks has detrimental effects on almost every aspect of the service.

Figure 9:

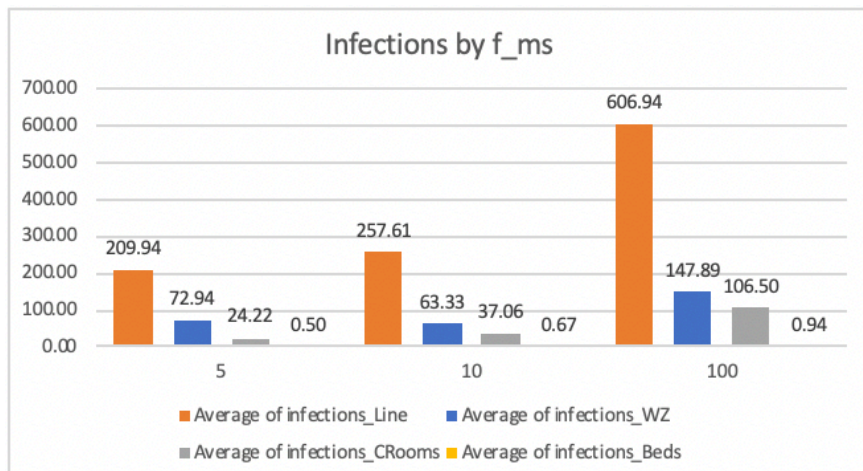
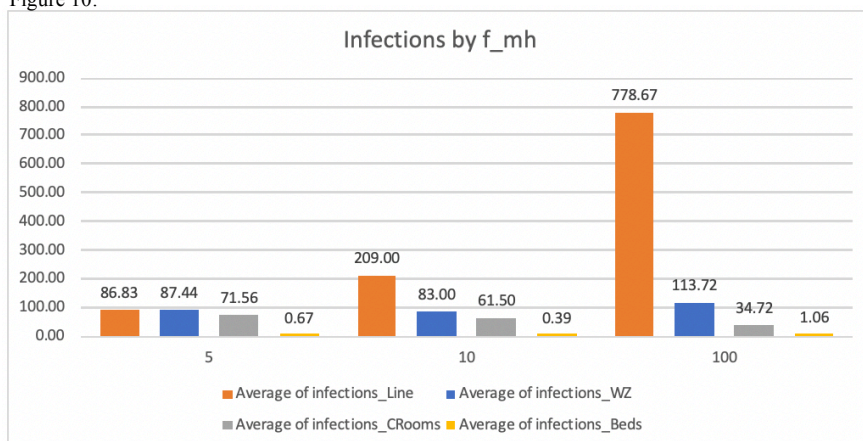


Figure 10:



5.2 Quantifying the impact of different quantities of hosts

Total simulations: 144

BehaviourSpace Parameters:

HostProb = [6, 10, 15]

AnxietyLevelAvg = [1,5,10]

F_mh = [5]

F_ms = [5]

RecpetionTimeAvg = [3, 8]

ConsultingTimeAvg = [20, 30]

Rapid-test-at-entrance = [false, true]

The *HostProb* is the probability that a patient that enters the hospital carries the virus. Data from the experiments show that increasing the *HostProb* significantly increases the average percentage of infections which is expected.

Figure 11:

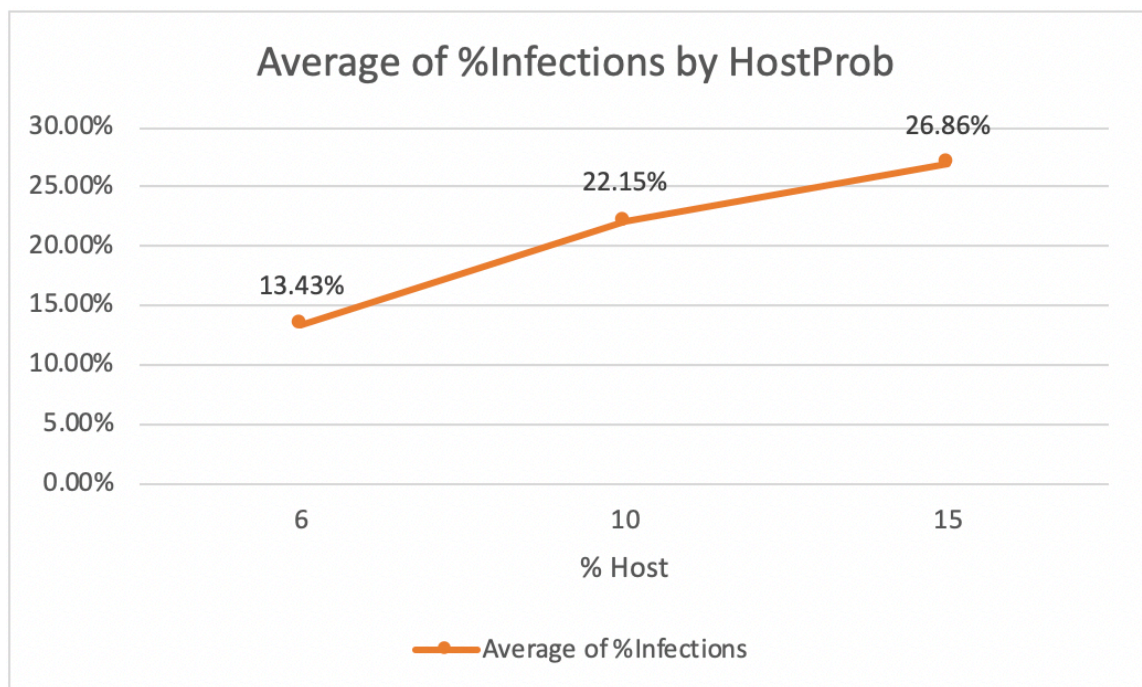
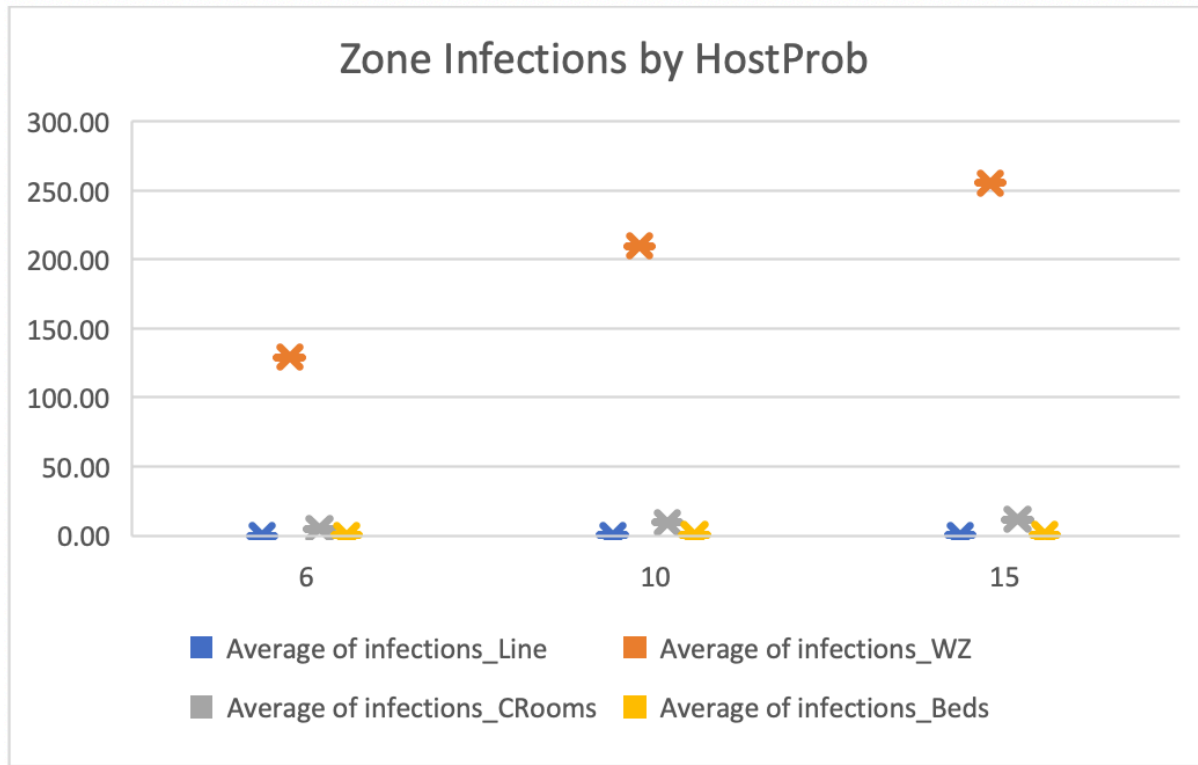


Figure 12:



5.3 Quantifying the impact of rapid-tests at the ER entrance

Total simulations: 198

BehaviourSpace Parameters:

HostProb = [6, 10, 15]

AnxietyLevelAvg = [1,5,10]

F_mh = [5, 10]

F_ms = [5, 10]

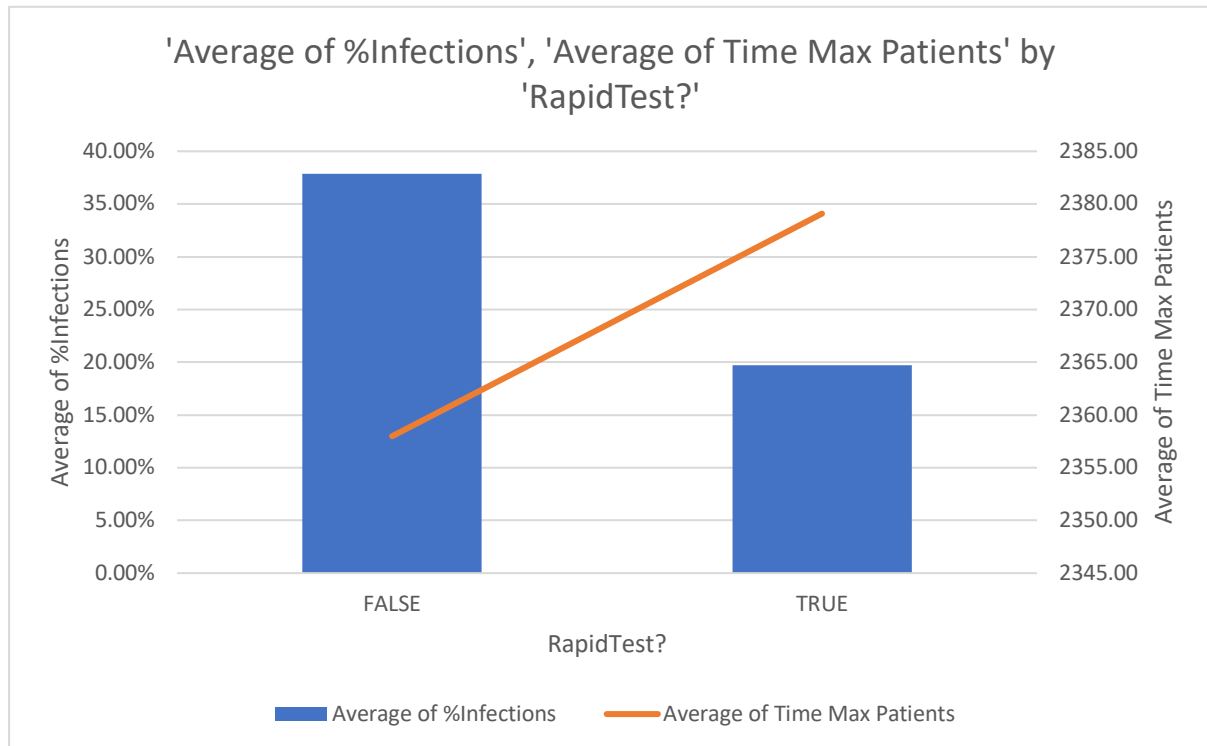
ReceptionTimeAvg = [3, 8]

ConsultingTimeAvg = [20, 30]

Rapid-test-at-entrance = [false, true]

Data from the experiments shows a massive improvement in patient infections when people are forced to take a rapid-test before being allowed to enter. The average number of infections dropped from 37.86% to 19.74% when the hospital implemented the measure.

Figure 13:



5.4 Quantifying the impact of social distancing measures and patient anxiety

Patient anxiety directly impacts social distancing. If patients are more jittery while waiting, resulting in them moving around more, then there will be more opportunities for contact between hosts and susceptible. This experiment was run with several different parameters in order to get a good idea of how anxiety levels impact infections. As it can be seen in the diagram below, anxiety levels from Level 1 to level 5 have little change in infections in any of the locations in the ED. However, an anxiety level of 10 increases the average number of infections in the run from 202.19 to 231.53 in the waiting room. This is likely attributed to the fact that the waiting room is where patients gather together.

Total simulations: 108

BehaviourSpace Parameters:

HostProb = [6, 10, 15]

AnxietyLevelAvg = [1, 5, 10]

F_mh = [5]

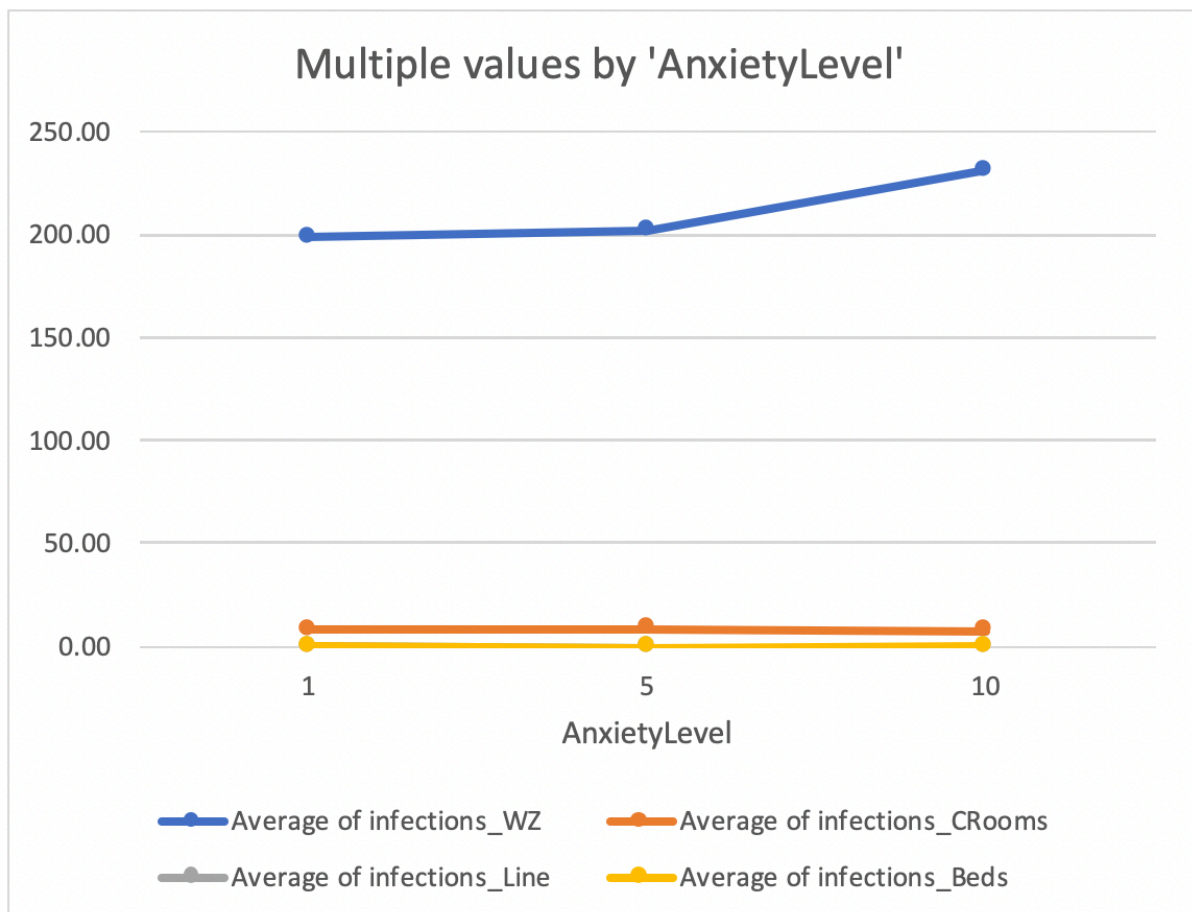
F_ms = [5]

RecpetionTimeAvg = [3]

ConsultingTimeAvg = [20, 30]

Rapid-test-at-entrance = [false, true]

Figure 14:



5.5 Quantifying the impact of waiting times

Data from the experiments shows that as waiting times for the reception and consultation times for patients increases, infections also increase. This is likely due to patients waiting around longer and hence breathing in more virions from nearby hosts. Interestingly, data shows that consultation times longer than 20 minutes greatly increase infections while increased reception times do not have a significant impact.

Total simulations: 72

BehaviourSpace Parameters:

HostProb = [6, 10, 15]

AnxietyLevelAvg = [5]

F_mh = [5]

F_ms = [5]

ReceptionTimeAvg = [3, 8]

ConsultingTimeAvg = [20, 30]

Rapid-test-at-entrance = [false, true]

Figure 15:

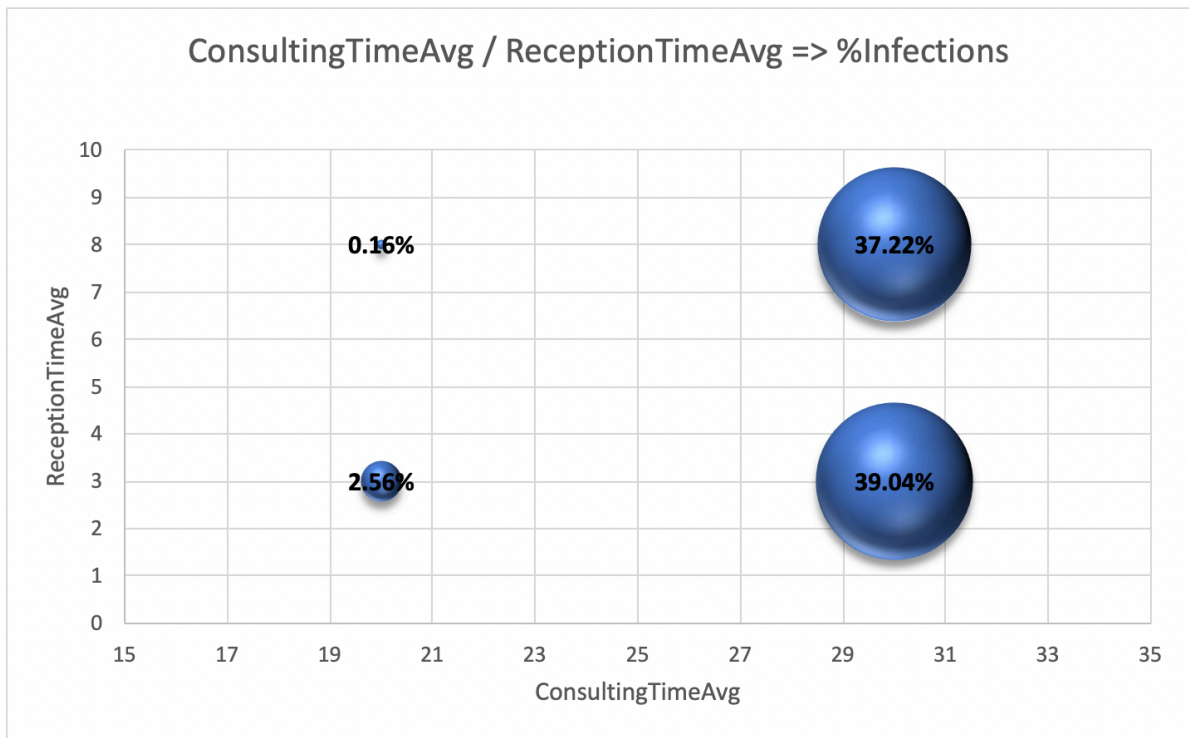


Figure 16:

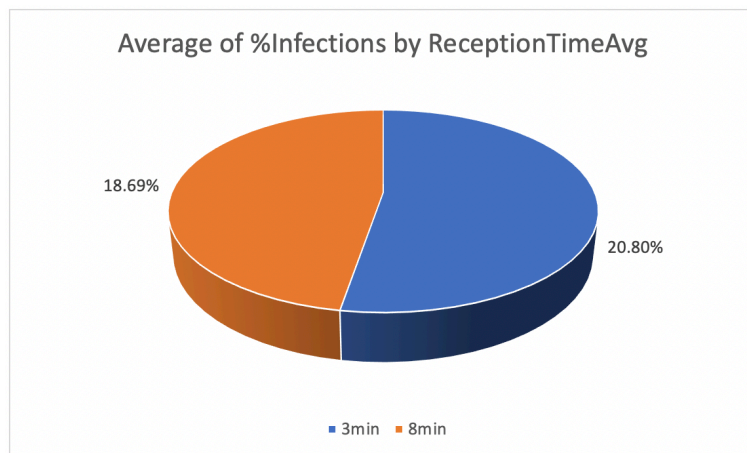
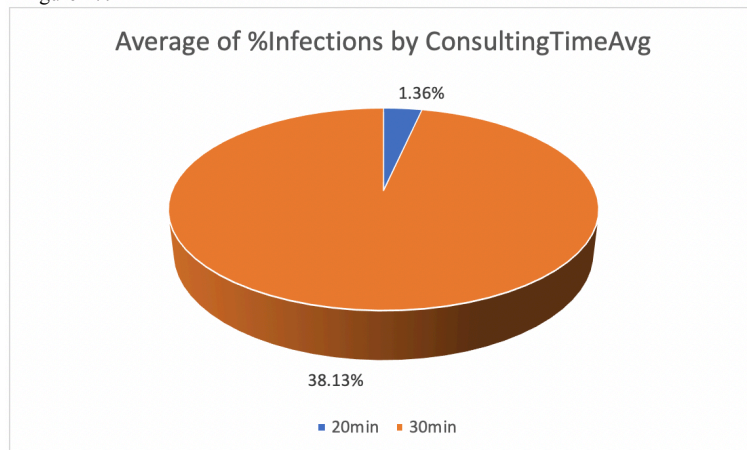


Figure 17:



Chapter 6

Legal, Social, Ethical and Professional Issues

6.1 Introduction

As discussed in the previous chapters, modelling the spread of viruses in an emergency room is a complex task that requires careful consideration of several factors. However, it is also imperative to consider the legal, social, ethical, and professional issues that may arise when conducting such modelling studies following the British Computer Society (BCS) code of conduct. This chapter will discuss these issues and their implications for the research conducted.

6.2 Legal Issues

When modelling the spread of viruses, it is essential to consider the legal implications of the research following the BCS code of conduct. Researchers must ensure their work adheres to all relevant laws and regulations, including patient confidentiality and privacy. The use of patient data in research must be conducted following the General Data Protection Regulation (GDPR) and other applicable laws and regulations. Researchers must obtain the necessary permissions and approvals before accessing patient data and ensure that the data is securely stored and used only for the purposes for which it was collected. In addition, researchers must be aware of the potential for their models to be used in legal proceedings following the BCS code of conduct. If their models are used as evidence in a legal case, researchers may be called upon to testify and defend their work. As such, it is essential to ensure that the research is conducted rigorously and transparently to minimise the risk of legal challenges.

4.3 Social Issues

Modelling the spread of viruses in an emergency room can also raise social issues in accordance with the BCS code of conduct. This research may have implications for the general public in the context of public health policy. Hence, it is of the highest importance that there is full transparency in the source code and methods used in the simulation so that the different readers are as well informed as possible. Researchers developing such simulations must ensure that their work is communicated clearly and effectively to a wide range of audiences, including policymakers and healthcare professionals. Researchers must also be aware of the potential for their work to be misinterpreted or misused by those with vested interests. Moreover, researchers must ensure that their models do not perpetuate existing social inequalities or biases in accordance with the BCS code of conduct. For example, certain populations may be more vulnerable to the spread of viruses due to socioeconomic or other factors. Researchers must ensure that their models take these factors into account and do not further exacerbate existing disparities.

4.4 Ethical Issues

Modelling the spread of viruses in an emergency department raises numerous ethical issues in accordance with the BCS code of conduct. Researchers must consider the potential risks and measure them against the benefits in accordance with the BCS code of conduct. While modelling the spread of viruses can provide valuable insights into public health policy, it may also have unintended consequences. For example, it may lead to the stigmatization of certain populations or the over-reliance on models over other forms of evidence. Further, there is always the case that the work will be used for malicious reasons. While these models can easily help develop strategies to reduce infections, it is also a possibility for the opposite, where they are used to examine how to inflict the most transmissions. If an actual model of this kind was being used to dictate policy, it would be crucial for it not to fall into the wrong hands.

4.5 Professional Issues

Finally, modelling the spread of viruses in an emergency room raises professional issues for researchers in accordance with the BCS code of conduct. Researchers must ensure that their

work adheres to the highest standards of scientific integrity and rigor. They must also ensure that they are transparent about their methods and data sources, and that they make their research accessible to other researchers in the field. Throughout this research, it was always a priority to use reputable sources of high confidence.

Chapter 7

Results/Evaluation

The objective of this project was to build an ABM simulation which can help hospital management make decisions about their services during airborne virus epidemics to limit the spread of the virus to other patients within the Emergency Department. The simulation has these capabilities, and in the experiments chapter, several observations were discussed. For example, the simulation demonstrated that most infections occur in the waiting room. Although the result data of the experimentation section were discussed briefly as this was not the focus of the research paper, a lot of time was spent analysing the data to verify the simulation was presenting valid outputs given the changes made to parameters. It was particularly interesting to see that the first time the CAT inequality was implemented, the simulation ran exactly in accordance to what one would deduce using common knowledge. All that there were essentially never any cases of something like people wearing more effective masks and that increasing the infections. Essentially there were no abnormalities in the expected outcomes which is very good sign for the model used. For those reasons, this project was a success.

The user is meant to have the ability to control several parameters described in the requirements chapters and access informative statistics about the ER. These requirements were also met. Further, the objective was also to use the novel CAT inequality as the contagion submodel to demonstrate the novel framework's potential when applied to agent-based models of indoor virus transmission simulations. This is the aspect of the project that excelled the most. The CAT inequality has three categories of parameters, host, susceptible and environment. We effectively assigned each of the stated parameters as agent properties and kept the environment properties as constants.

Finally, the objective from the very start was to develop a simulation that is easy to use and user-friendly. Following the User Guide in Appendix B, one can get to playing around with different scenarios in under 10 minutes. The objective was always to avoid making a system that takes days to set-up and runs very slowly. So, in this aspect, the project is a major success.

Chapter 8

Conclusion and Future Work

Following the discussion in Chapter 7, it is evident that using the CAT inequality framework in conjunction with ABM yields a fascinating and promising model of indoor virus transmissions. This project focused on demonstrating that combination rather than determining particular intervention strategies for emergency departments. That would be a task suited for a more refined model, and the research should be done by virologists in combination with computer scientists. Although dozens of runs were conducted on the ED in the experimentation section, and some data were analysed to showcase the simulation's functioning and soundness, the virus and general conditions did not aim to mimic any existing virus. Therefore, it is not that the simulation results are valid but that the simulation as a system is valid. Given more time, or in future work, it would have been fascinating to research further the data generated by the simulation with the support of virologists, given that infectious diseases are not my field of expertise. It would also be fascinating if someone built on this work by dedicating a full research paper focused on the results of intervention strategies. It would have also been stimulating to go beyond the Stavanger University ED map to compare results over a series of departments and draw conclusions as to which factors significantly impact the average number of transmissions. It would have been interesting to change the layout into several versions within the same ED to analyse what factors bottleneck patient flow or lead to infections.

Regarding the actual simulation, although the final version is quite suitable for what we aimed to achieve, given more time, some improvements could be made, such as refining the flow of patients within the map even further so that their movements are as natural as possible. In a more advanced modelling platform, using some artificial intelligence would go a long way to model real-life people as much as possible. Further, the simulation could be of even greater scientific value if it used the medical data of actual patients in the region of the ER in order to simulate the kinds of agents better. However, as discussed in Chapter 6, this would likely not be readily allowed, given privacy regulations.

References

Asadi, S., Cappa, C.D., Barreda, S., Wexler, A.S., Bouvier, N.M. and Ristenpart, W.D. (2020). Efficacy of masks and face coverings in controlling outward aerosol particle emission from expiratory activities. *Scientific Reports*, [online] 10(1), p.15665. doi:<https://doi.org/10.1038/s41598-020-72798-7>.

Asadi, S., Wexler, A.S., Cappa, C.D., Barreda, S., Bouvier, N.M. and Ristenpart, W.D. (2019). Aerosol emission and superemission during human speech increase with voice loudness. *Scientific Reports*, 9(1). doi:<https://doi.org/10.1038/s41598-019-38808-z>.

Bagni, R. ; B. (2002). A comparison of simulation models applied to epidemics. [online] jasss.soc.surrey.ac.uk. Available at: <https://jasss.soc.surrey.ac.uk/5/3/5.html> [Accessed 6 Apr. 2023].

Belser, J.A., Gustin, K.M., Pearce, M.B., Maines, T.R., Zeng, H., Pappas, C., Sun, X., Carney, P.J., Villanueva, J.M., Stevens, J., Katz, J.M. and Tumpey, T.M. (2013). Pathogenesis and transmission of avian influenza A (H7N9) virus in ferrets and mice. *Nature*, [online] 501(7468), pp.556–559. doi:<https://doi.org/10.1038/nature12391>.

Bourouiba, L., Dehandschoewercker, E. and Bush, John W. M. (2014). Violent expiratory events: on coughing and sneezing. *Journal of Fluid Mechanics*, [online] 745, pp.537–563. doi:<https://doi.org/10.1017/jfm.2014.88>.

Brauer, F., Driessche, P. van den and Wu, J. (2008). *Mathematical Epidemiology*. [online] Google Books. Springer Science & Business Media. Available at: https://www.google.co.uk/books/edition/Mathematical_Epidemiology/gcP511a22rQC?hl=en&gbpv=1&dq=agent+based+modeling+for+viruses&printsec=frontcover.

builtin.com. (n.d.). Empirical Rule (68-95-99.7) Explained | Built In. [online] Available at: <https://builtin.com/data-science/empirical-rule>.

Chen, Y.-C., Huang, L.-M., Chan, C.-C., Su, C.-P., Chang, S.-C., Chang, Y.-Y., Chen, M.-L., Hung, C.-C., Chen, W.-J., Lin, F.-Y. and Lee, Y.-T. (2004). SARS in Hospital Emergency Room. *Emerging Infectious Diseases*, [online] 10(5), pp.782–788. doi:<https://doi.org/10.3201/eid1005.030579>.

Cheng, Y., Han, L., Wu, L., Chen, J., Sun, H., Wen, G., Ji, Y., Dvorkin, M., Shi, J., Pan, Z., Shi, J., Wang, X., Bai, Y., Melkadze, T., Pan, Y., Min, X., Viguro, M., Li, X., Zhao, Y. and Yang, J. (2022). Effect of First-Line Serplulimab vs Placebo Added to Chemotherapy on Survival in Patients With Extensive-Stage Small Cell Lung Cancer: The ASTRUM-005 Randomized Clinical Trial. *JAMA*, [online] 328(12), pp.1223–1232. doi:<https://doi.org/10.1001/jama.2022.16464>.

- Drossinos, Y. and Stilianakis, N.I. (2020). What aerosol physics tells us about airborne pathogen transmission. *Aerosol Science and Technology*, 54(6), pp.639–643. doi:<https://doi.org/10.1080/02786826.2020.1751055>.
- Edwards, D.A., Man, J.C., Brand, P., Katstra, J.P., Sommerer, K., Stone, H.A., Nardell, E. and Scheuch, G. (2004). Inhaling to mitigate exhaled bioaerosols. *Proceedings of the National Academy of Sciences of the United States of America*, [online] 101(50), pp.17383–17388. doi:<https://doi.org/10.1073/pnas.0408159101>.
- Ehsanifar, M. (2021). Airborne aerosols particles and COVID-19 transition. *Environmental Research*, [online] 200, p.111752. doi:<https://doi.org/10.1016/j.envres.2021.111752>.
- Gammage, R.B. (1996). *Indoor Air and Human Health*. [online] Google Books. CRC Press. Available at: https://www.google.co.uk/books/edition/Indoor_Air_and_Human_Health/rL1ue-pMhrAC?hl=en&gbpv=0.
- Giallongo, C., Tibullo, D., Camiolo, G., Parrinello, N.L., Romano, A., Puglisi, F., Barbato, A., Conticello, C., Lupo, G., Anfuso, C.D., Lazzarino, G., Li Volti, G., Palumbo, G.A. and Di Raimondo, F. (2019). TLR4 signaling drives mesenchymal stromal cells commitment to promote tumor microenvironment transformation in multiple myeloma. *Cell Death & Disease*, 10(10). doi:<https://doi.org/10.1038/s41419-019-1959-5>.
- Gualano, M.R., Sinigaglia, T., Lo Moro, G., Rousset, S., Cremona, A., Bert, F. and Siliquini, R. (2021). The Burden of Burnout among Healthcare Professionals of Intensive Care Units and Emergency Departments during the COVID-19 Pandemic: A Systematic Review. *International Journal of Environmental Research and Public Health*, [online] 18(15). doi:<https://doi.org/10.3390/ijerph18158172>.
- Guardian, T. (2021). Up to 8,700 patients died after catching Covid in English hospitals. [online] the Guardian. Available at: <https://www.theguardian.com/world/2021/may/24/up-to-8700-patients-died-after-catching-covid-in-english-hospitals>.
- Hafner, K. (2020). Fear of Covid-19 Leads Other Patients to Decline Critical Treatment. *The New York Times*. [online] 29 May. Available at: <https://www.nytimes.com/2020/05/25/health/coronavirus-cancer-heart-treatment.html>.
- Hardy, K.J., Oppenheim, B.A., Gossain, S., Gao, F. and Hawkey, P.M. (2006). A Study of the Relationship Between Environmental Contamination with Methicillin-Resistant *Staphylococcus aureus* (MRSA) and Patients' Acquisition of MRSA. *Infection Control & Hospital Epidemiology*, 27(2), pp.127–132. doi:<https://doi.org/10.1086/500622>.
- Hayden, M.K., Blom, D.W., Lyle, E.A., Moore, C.G. and Weinstein, R.A. (2008). Risk of Hand or Glove Contamination After Contact With Patients Colonized With Vancomycin-Resistant *Enterococcus* or the Colonized Patients' Environment. *Infection Control & Hospital Epidemiology*, 29(2), pp.149–154. doi:<https://doi.org/10.1086/524331>.

Hethcote, H.W. (2000). The Mathematics of Infectious Diseases. *SIAM Rev.*, [online] 42(4), pp.599–653. doi:<https://doi.org/10.1137/s0036144500371907>.

Hunter, E., Mac Namee, B. and Kelleher, J. (2018). An open-data-driven agent-based model to simulate infectious disease outbreaks. *PLOS ONE*, 13(12), p.e0208775. doi:<https://doi.org/10.1371/journal.pone.0208775>.

Keeling, M.J. and Rohani, P. (2008). *Modeling Infectious Diseases in Humans and Animals*. [online] JSTOR. Princeton University Press. Available at: <https://www.jstor.org/stable/j.ctvcn4gk0>.

Kumar, D. and Dey, T. (2020). Treatment delays in oncology patients during COVID-19 pandemic: A perspective. *Journal of Global Health*, 10(1). doi:<https://doi.org/10.7189/jogh.10.010367>.

Leung, N.H.L. (2021). Transmissibility and transmission of respiratory viruses. *Nature Reviews Microbiology*, [online] 19, pp.1–18. doi:<https://doi.org/10.1038/s41579-021-00535-6>.

Lindsley, W.G., Pearce, T.A., Hudnall, J.B., Davis, K.A., Davis, S.M., Fisher, M.A., Khakoo, R., Palmer, J.E., Clark, K.E., Celik, I., Coffey, C.C., Blachere, F.M. and Beezhold, D.H. (2012). Quantity and Size Distribution of Cough-Generated Aerosol Particles Produced by Influenza Patients During and After Illness. *Journal of Occupational and Environmental Hygiene*, [online] 9(7), pp.443–449. doi:<https://doi.org/10.1080/15459624.2012.684582>.

Mandavilli, A. (2020). How Long Will Coronavirus Live on Surfaces or in the Air Around You? *The New York Times*. [online] 17 Mar. Available at: <https://www.nytimes.com/2020/03/17/health/coronavirus-surfaces-aerosols.html>.

Morawska, L., Johnson, G.R., Ristovski, Z.D., Hargreaves, M., Mengersen, K., Corbett, S., Chao, C.Y.H., Li, Y. and Katoshevski, D. (2009). Size distribution and sites of origin of droplets expelled from the human respiratory tract during expiratory activities. *Journal of Aerosol Science*, 40(3), pp.256–269. doi:<https://doi.org/10.1016/j.jaerosci.2008.11.002>.

Nicas, M., Nazaroff, W.W. and Hubbard, A. (2005). Toward Understanding the Risk of Secondary Airborne Infection: Emission of Respirable Pathogens. *Journal of Occupational and Environmental Hygiene*, 2(3), pp.143–154. doi:<https://doi.org/10.1080/15459620590918466>.

Nigel, G. and Klaus, T. (2005). *Simulation For The Social Scientist*. [online] Google Books. McGraw-Hill Education (UK). Available at: https://books.google.co.uk/books/about/Simulation_For_The_Social_Scientist.html?id=fBlaulpmNowC&redir_esc=y.

Otter, J.A., Yezli, S., Salkeld, J.A.G. and French, G.L. (2013). Evidence that contaminated surfaces contribute to the transmission of hospital pathogens and an overview of strategies to address contaminated surfaces in

hospital settings. *American Journal of Infection Control*, 41(5), pp.S6–S11.
doi:<https://doi.org/10.1016/j.ajic.2012.12.004>.

Railsback, S.F. and Grimm, V. (2011). *Agent-Based and Individual-Based Modeling: A Practical Introduction*. [online] JSTOR. Princeton University Press. Available at: <https://www.jstor.org/stable/j.ctt7sns7> [Accessed 5 Apr. 2023].

Siegel, J.D., Rhinehart, E., Jackson, M. and Chiarello, L. (2007). 2007 Guideline for Isolation Precautions: Preventing Transmission of Infectious Agents in Health Care Settings. *American Journal of Infection Control*, [online] 35(10), pp.S65–S164. doi:<https://doi.org/10.1016/j.ajic.2007.10.007>.

Siettos, C.I. and Russo, L. (2013). Mathematical modeling of infectious disease dynamics. *Virulence*, [online] 4(4), pp.295–306. doi:<https://doi.org/10.4161/viru.24041>.

Sze To, G.N. and Chao, C.Y.H. (2010). Review and comparison between the Wells-Riley and dose-response approaches to risk assessment of infectious respiratory diseases. *Indoor Air*, [online] 20(1), pp.2–16. doi:<https://doi.org/10.1111/j.1600-0668.2009.00621.x>.

Wagenvoort, J.H.T., Sluijsmans, W. and Penders, R.J.R. (2000). Better environmental survival of outbreak vs. sporadic MRSA isolates. *Journal of Hospital Infection*, 45(3), pp.231–234. doi:<https://doi.org/10.1053/jhin.2000.0757>.

Wang, L. (n.d.). Comparison of Objects and Agents. [online] Available at: https://people.ucalgary.ca/~far/Lectures/SENG697/PDF/tutorials/2002/Comparison_of_Objects_and_Agents.pdf.

Jacobs, Samantha E., et al. “Human Rhinoviruses.” *Clinical Microbiology Reviews*, vol. 26, no. 1, Jan. 2013, pp. 135–162, cmr.asm.org/content/26/1/135, <https://doi.org/10.1128/cmr.00077-12>.

Bagni, R. ; Berchi. “A Comparison of Simulation Models Applied to Epidemics.” *Jasss.soc.surrey.ac.uk*, 30 June 2002, jasss.soc.surrey.ac.uk/5/3/5.html.

www.scirp.org. (n.d.). Bonabeau, E. (2002) Agent-Based Modeling Methods and Techniques for Simulating Human Systems. *Proceedings of the National Academy of Sciences of the United States of America*, 99, 7280–7287. - References - Scientific Research Publishing. [online] Available at: [https://www.scirp.org/\(S\(351jmbntvnsjt1aadkposzje\)\)/reference/ReferencesPapers.aspx?ReferenceID=1598418](https://www.scirp.org/(S(351jmbntvnsjt1aadkposzje))/reference/ReferencesPapers.aspx?ReferenceID=1598418).